HEPTA CODE

Specifica Tecnica

Versione 1.0.0

Contenuto del Documento

Il presente documento contiene la specifica tecnica del gruppo *Hepta Code* relativo al capitolato *Code Guardian* proposto dall'azienda *Var Group*.

email: heptacode7@gmail.com

sito: heptacode-unipd.github.io

Registro delle modifiche

Versione	Data	Autore	Verificatore	Descrizione
1.0.0	2026/04/07	Angela Favaro	Laura Venturini	Approvazione finale, rilascio ufficiale v1.0.0
0.7.0	2026/04/07	Amerigo Vegliante	Laura Venturini	Concluso specifiche MS1
0.6.0	2026/04/07	Angela Canazza	Angela Favaro	Concluso specifiche MS0
0.5.0	2026/04/03	Alberto Reginato	Riccardo Baldin	Aggiunte specifiche MS2
0.4.0	2026/04/01	Angela Favaro	Nicola Simionato	Aggiunte specifiche MS3
0.3.0	2026/04/01	Angela Canazza	Angela Favaro	Aggiunte specifiche MS0
0.2.0	2026/03/08	Angela Favaro	Angela Canazza	Aggiunto capitolo 1
0.1.0	2026/03/07	Angela Favaro	Laura Venturini	Creazione documento, impostazione macro-aree

Indice dei contenuti

1. Introduzione ... 1
   1.1. Scopo del documento ... 1
   1.2. Glossario ... 1
   1.3. Riferimenti ... 1
      1.3.1. Riferimenti Normativi ... 1
      1.3.2. Riferimenti Informativi ... 1
2. Tecnologie ... 2
   2.1. Linguaggi di programmazione ... 2
      2.1.1. Typescript v5.7.x ... 2
   2.2. Framework ... 2
      2.2.1. NestJS v11.0.x ... 2
   2.3. Librerie e dipendenze ... 2
      2.3.1. React v19.2.0 ... 2
      2.3.2. Dipendenze frontend (React) ... 3
      2.3.3. Dipendenze backend (Node.js / NestJS) ... 5
   2.4. Runtime environment ... 7
      2.4.1. NodeJS v24 ... 7
   2.5. Infrastruttura di deployment ... 8
      2.5.1. Docker Engine v29.3.0 ... 8
   2.6. Tecnologie per la persistenza dei dati ... 8
      2.6.1. MongoDB v8.0.x ... 8
   2.7. Tecnologie per infrastruttura cloud ... 8
      2.7.1. AWS ... 8
         2.7.1.1. AWS Bedrock ... 8
         2.7.1.2. AWS Lambda ... 9
         2.7.1.3. AWS Step Functions ... 9
         2.7.1.4. AWS S3 ... 9
         2.7.1.5. AWS Fargate ... 9
   2.8. Tecnologie per Continuous Integration ... 9
      2.8.1. GitHub Actions ... 9
3. Architettura ... 9
   3.1. Architettura logica ... 10
      3.1.1. Layered Architecture ... 10
   3.2. Architettura di deployment ... 10
      3.2.1. Microservizi ... 11
   3.3. Design pattern ... 11
4. Progettazione ... 12
   4.1. Progettazione backend ... 12
      4.1.1. Analysis Management Service - MS1 ... 12
         4.1.1.1. Classi MS1 - Presentation Layer ... 13
         4.1.1.2. Classi MS1 - Business Layer ... 14
         4.1.1.3. Classi MS1 - Infrastructure Layer ... 14
         4.1.1.4. Classi MS1 - Data Layer ... 14
         4.1.1.5. MS1 - Workflow di Comunicazione ... 15
      4.1.2. Analisi dei Repository - MS2 ... 16
         4.1.2.1. Classi MS2 - Presentation Layer ... 16
         4.1.2.2. Classi MS2 - Business Layer ... 17

4.1.2.2.1. Orchestrazione e Workflow ... 17
4.1.2.2.2. Agenti di Analisi Tematica ... 17
4.1.2.3. Classi MS2 - Data Layer ... 18
4.1.2.4. MS2 - Workflow di Orchestrazione ... 18
4.1.2.4.1. Strumenti e Utility ... 19
4.1.3. Autenticazione e Repository Management - MS3 ... 21
4.1.3.1. Classi MS3 - Presentation Layer ... 22
4.1.3.2. Classi MS3 - Business Layer ... 23
4.1.3.2.1. Interfacce ... 23
4.1.3.2.2. Servizi ... 24
4.1.3.2.3. Entità di dominio ... 25
4.1.3.3. Classi MS3 - Data Layer ... 25
4.2. Progettazione frontend ... 27
4.2.1. Moduli MS0 - Presentation Layer ... 29
4.2.1.1. Routing ... 29
4.2.1.2. Componenti ... 29
4.2.1.3. Pagine ... 30
4.2.2. Moduli MS0 - Business Layer ... 33
4.2.3. Moduli MS0 - Data Layer ... 35
4.2.3.1. HTTP Client ... 35
4.2.3.2. Tipi di dominio ... 36
4.2.3.3. Configurazione ... 36
5. Stato dei requisiti funzionali ... 36
5.1. Stato per requisito ... 36
5.2. Grafici di stato dei requisiti ... 47
5.2.1. Requisiti funzionali ... 47
5.2.2. Requisiti funzionali obbligatori ... 50
5.2.3. Requisiti funzionali desiderabili ... 50
5.2.4. Requisiti funzionali opzionali ... 50

Indice delle tabelle

Tabella 1 Dipendenze frontend, core ... 3
Tabella 2 Dipendenze frontend, routing e utility ... 3
Tabella 3 Dipendenze frontend, build e compilazione ... 3
Tabella 4 Dipendenze frontend, testing ... 3
Tabella 5 Dipendenze frontend, qualità del codice ... 4
Tabella 6 Dipendenze backend, framework e core ... 5
Tabella 7 Dipendenze backend, persistenza ... 5
Tabella 8 Dipendenze backend, sicurezza ... 5
Tabella 9 Dipendenze backend, validazione ... 5
Tabella 10 Dipendenze backend, testing ... 6
Tabella 11 Dipendenze backend, qualità del codice ... 6
Tabella 12 Dipendenze backend, integrazione cloud AWS ... 6
Tabella 13 Dipendenze backend, documentazione ... 7
Tabella 14 Dipendenze backend, utility di analisi ... 7
Tabella 15 Dipendenze backend, infrastruttura serverless ... 7
Tabella 16 Dipendenze backend, integrazione VCS ... 7
Tabella 17 Dipendenze backend, comunicazione HTTP ... 7
Tabella 18 Copertura requisiti funzionali ... 36

Indice delle figure

Figura 1 Diagramma delle classi; Analisi Management Service - MS1 ... 13
Figura 2 Presentation Layer MS2 ... 17
Figura 3 Workflow di orchestrazione degli agenti tramite AWS Step Functions ... 19
Figura 4 Diagramma delle classi; Autenticazione - MS3 ... 21
Figura 5 Diagramma delle classi; Repository Management - MS3 ... 22
Figura 6 Layered Architecture - frontend ... 28
Figura 7 Diagramma componenti, App - frontend ... 29
Figura 8 Diagramma componenti, Login - frontend ... 30
Figura 9 Diagramma componenti, Repositories- frontend ... 31
Figura 10 Diagramma componenti, DettagliRepo- frontend ... 31
Figura 11 Diagramma componenti, AddRepository- frontend ... 32
Figura 12 Diagramma componenti, UserPage - frontend ... 32
Figura 13 Diagramma componenti, SessionService - frontend ... 33
Figura 14 Diagramma componenti, UserService - frontend ... 33
Figura 15 Diagramma componenti, RepositoriesService - frontend ... 34
Figura 16 Diagramma componenti, AnalysisService - frontend ... 35
Figura 17 Requisiti funzionali soddisfatti ... 47
Figura 18 Requisiti funzionali soddisfatti Utente generico ... 47
Figura 19 Requisiti funzionali soddisfatti Developer ... 48
Figura 20 Requisiti funzionali soddisfatti Project Manager ... 48
Figura 21 Requisiti funzionali soddisfatti Business Owner ... 49
Figura 22 Requisiti funzionali obbligatori soddisfatti ... 50
Figura 23 Requisiti funzionali desiderabili soddisfatti ... 50
Figura 24 Requisiti funzionali opzionali soddisfatti ... 51

1. Introduzione

1.1. Scopo del documento

Il documento di Specifica Tecnica descrive come il sistema software[G] deve essere progettato e realizzato per soddisfare i requisiti all'interno del documento di *Analisi dei Requisiti v3.0.0*.
Le funzioni del seguente documento sono:
- Guida durante l'implementazione, riducendo ambiguità e decisioni improvvisate.
- Allineamento del team su scelte architetturali e tecnologiche.
- Supporto alla manutenzione futura.
- Agevolazione dell'attività di testing.

1.2. Glossario

La stesura del presente documento fa uso di una terminologia specifica, legata sia al dominio applicativo del progetto «*Code Guardian*» che agli standard dell'Ingegneria del Software. Per facilitare la lettura e assicurare che ogni concetto sia compreso in modo uniforme da tutti i destinatari, è stato redatto un documento di supporto dedicato.

Si faccia pertanto riferimento al *Glossario v3.0.0* per l'esplicitazione di:
- **Acronimi e sigle** utilizzati per brevità nel testo;
- **Termini tecnici** che potrebbero prestarsi a molteplici interpretazioni;
- **Definizioni di dominio** specifiche per il contesto di audit[G] e qualità del software.

L'uso del glossario è raccomandato per garantire la piena coerenza semantica durante tutte le fasi del ciclo di vita del progetto.

1.3. Riferimenti

Questa sezione elenca i documenti utilizzati come base per la stesura della presente analisi.

1.3.1. Riferimenti Normativi

- *Code Guardian*: Piattaforma ad agenti per l'audit e la remediation[G] dei repository[G] software (Capitolato)
- *Norme di progetto*: Regole, standard e procedure del gruppo *Hepta Code* (Norme di progetto v2.0.0).

1.3.2. Riferimenti Informativi

- Regolamento del progetto didattico: Regolamento di progetto
- Lezioni su progettazioni e pattern architetturali:
  - Progettazione e programmazione: Diagrammi delle classi (UML)
  - Progettazione: I pattern architetturali
  - Progettazione: Il pattern Dependency Injection
  - Progettazione: I pattern strutturali
  - Progettazione: I pattern di comportamento
- Documento interno: Glossario v3.0.0

Questa introduzione delinea il contesto e gli scopi del progetto.

2. Tecnologie

2.1. Linguaggi di programmazione

2.1.1. Typescript v5.7.x

Il progetto è sviluppato in Typescript, un superset di JavaScript che introduce tipizzazione statica opzionale.
La **tipizzazione statica e riduzione degli errori a runtime** TypeScript consente di intercettare intere categorie di errori già in fase di compilazione, prima che il codice raggiunga l'ambiente di esecuzione.
In un'applicazione full-stack dove frontend[G] e backend[G] si scambiano dati via API, la definizione di interfacce e tipi condivisi elimina ambiguità sui contratti di dato, riducendo drasticamente i bug dovuti a proprietà mancanti, tipi inattesi o refactoring parziali.
Il supporto al completamento automatico, alla navigazione del codice e al refactoring assistito offerto dagli IDE come VS Code è notevolmente potenziato dalla presenza dei tipi. Questo si traduce in **cicli di sviluppo più rapidi e in un onboarding più agevole per nuovi membri del team**.
Su tutti gli strati applicativi **è possibile condividere modelli di dominio, DTO (Data Transfer Object) e interfacce in un unico package condiviso**, garantendo coerenza tra le strutture dati prodotte dal backend e quelle consumate dal frontend, senza duplicazione del codice.

2.2. Framework[G]

2.2.1. NestJS v11.0.x

NestJS è il framework applicativo scelto per strutturare il layer server[G].

- *Architettura modulare e scalabile*: organizza il codice in moduli, controller e service, imponendo una struttura chiara che facilita la separazione delle responsabilità e la crescita ordinata del progetto nel tempo.
- *Dependency Injection nativa*: Il sistema di DI integrato rende i componenti disaccoppiati, testabili in isolamento e facilmente sostituibili, migliorando la qualità architetturale complessiva.
- *NestJS è scritto nativamente in TypeScript*: ne sfrutta appieno i decoratori e il sistema di tipi, rendendolo la scelta più coerente con il resto della stack.
- *Supporto integrato per pattern enterprise*: NestJS offre supporto per pipe di validazione, middleware e gestione centralizzata degli errori, riducendo la necessità di soluzioni «su misura» per funzionalità trasversali.

2.3. Librerie e dipendenze

2.3.1. React v19.2.0

React è stato scelto come libreria[G] UI[G], le motivazioni principali sono state:

- *Modello a componenti* - L'architettura basata su componenti riutilizzabili favorisce la separazione delle responsabilità, facilita i test unitari e permette di costruire interfacce complesse in modo incrementale e controllato.
- *Ecosistema maturo* - React dispone di un ecosistema vastissimo che spesso non richiede soluzioni «su misura».
- *Virtual DOM e performance* - Il meccanismo di riconciliazione del Virtual DOM garantisce aggiornamenti efficienti dell'interfaccia, limitando le operazioni sul DOM reale ai[G] soli nodi effettivamente modificati.
- *Integrazione nativa con TypeScript* - React supporta pienamente TypeScript, con tipizzazione completa di props, state, hook e context, rendendo il codice frontend robusto e verificabile staticamente.

2.3.2. Dipendenze frontend (React)

Tabella 1: Dipendenze frontend, core

Nome	Versione	Descrizione
react, react-dom	v19.2.0	Libreria principale per la costruzione dell'interfaccia utente e il rendering nel DOM. Scelto per la sua architettura a componenti e il vasto ecosistema.

Tabella 2: Dipendenze frontend, routing e utility

Nome	Versione	Descrizione
react-router-dom	v7.13.1	Gestione del routing lato client. Fornisce i componenti <Route>, <Link> e gli hook useNavigate e useParams per la navigazione tra le pagine.
react-icons	v5.6.0	Libreria di icone SVG. Aggrega i principali icon set in un unico pacchetto importabile come componenti React.
react-markdown	v9.1.0	Renderizza contenuto Markdown come componenti React. Utilizzata per visualizzare i report delle analisi restituiti dal backend in formato Markdown.

Tabella 3: Dipendenze frontend, build e compilazione

Nome	Versione	Descrizione
vite	v7.2.4	Bundler e dev server. Scelto per la velocità di avvio grazie all'uso nativo dei moduli ES e all'HMR istantaneo.
@vitejs/plugin-react	v5.1.1	Plugin Vite che abilita il supporto a JSX e al Fast Refresh di React durante lo sviluppo.
typescript	v5.7.x	Superset tipizzato di JavaScript. Usato per garantire la correttezza dei tipi a tempo di compilazione.
@types/react, @types/react-dom, @types/node	varie	Definizioni dei tipi TypeScript per le rispettive librerie.

Tabella 4: Dipendenze frontend, testing

Nome	Versione	Descrizione
vitest	v4.1.1	Framework di test nativo per Vite. Compatibile con l'API di Jest, condivide la configurazione di Vite evitando setup duplicati.
@vitest/coverage-v8	v4.1.1	Provider[G] di code coverage[G] basato su V8. Attivato tramite il comando test:cov, non richiede import espliciti nel codice.
@vitest/ui	v4.1.1	Interfaccia grafica per visualizzare i risultati dei test nel browser[G].
@testing-library/react, @testing-library/dom	v16.3.2 / v10.4.1	Utility per testare componenti React simulando l'interazione reale con il DOM, senza dipendere dall'implementazione interna.
@testing-library/jest-dom	v6.9.1	Estende i matcher di Vitest con asserzioni specifiche per il DOM (toBeInTheDocument, toHaveClass, ecc.).
@testing-library/user-event	v14.6.1	Simula eventi utente realistici (click, digitazione, tab) nei test, più fedele al comportamento del browser rispetto a fireEvent.
jsdom	v29.0.1	Implementazione del DOM in ambiente Node.js[G]. Usato da Vitest come ambiente di test per simulare il browser.
msw	v2.12.14	Mock Service Worker. Intercetta le richieste HTTP a livello di rete per simulare le API nei test di integrazione.

Tabella 5: Dipendenze frontend, qualità del codice

Nome	Versione	Descrizione
eslint, @eslint/js	v9.39.1	Analisi statica del codice TypeScript.
typescript-eslint	v8.46.4	Plugin ESLint per TypeScript. Abilita regole type-aware che richiedono le informazioni del compilatore.

Nome	Versione	Descrizione
eslint-plugin-react-hooks	v7.0.1	Controlla il corretto utilizzo degli hook React (Rules of Hooks).
eslint-plugin-react-refresh	v0.4.24	Avverte se un componente non è compatibile con il Fast Refresh di Vite.
globals	v16.5.0	Fornisce le liste di variabili globali per i vari ambienti (browser, node) usate nella configurazione ESLint.

2.3.3. Dipendenze backend (Node.js / NestJS)

Tabella 6: Dipendenze backend, framework e core

Nome	Versione	Descrizione
@nestjs/common, @nestjs/core	v11.0.1	Gestisce il sistema di Dependency Injection, i moduli, i controller e i decorator. Ideale per microservizi strutturati.
@nestjs/platform-express	v11.0.1	Adattatore HTTP per NestJS basato su Express. Gestisce il server HTTP sottostante.

Tabella 7: Dipendenze backend, persistenza

Nome	Versione	Descrizione
@nestjs/mongoose	v11.0.4	Integrazione ufficiale NestJS con Mongoose. Fornisce i decorator @Schema, @Prop, @InjectModel per collegare il layer di persistenza al framework.
mongoose	v9.3.2	ODM per MongoDB[G]. Gestisce la definizione degli schemi, le query e la connessione al database. Scelto per la sua maturità e integrazione nativa con NestJS.
mongodb	v7.1.1	Driver nativo MongoDB, dipendenza di Mongoose.

Tabella 8: Dipendenze backend, sicurezza

Nome	Versione	Descrizione
bcrypt	v6.0.0	Hashing delle password. Usato nel layer applicativo per non salvare mai password in chiaro nel database.

Tabella 9: Dipendenze backend, validazione

Nome	Versione	Descrizione
class-validator	v0.15.1	Validazione dei DTO tramite decorator (@IsEmail, @IsUrl, @MinLength). Integrato con NestJS tramite ValidationPipe
class-transformer	v0.5.1	Trasformazione degli oggetti plain in istanze di classe. Necessario per far funzionare class-validator con NestJS.

Tabella 10: Dipendenze backend, testing

Nome	Versione	Descrizione
jest	v30.x	Framework di test principale
ts-jest	v29.x	Permette di eseguire test scritti in TypeScript direttamente senza compilazione separata.
@nestjs/testing	v11.0.1	Utility per creare moduli NestJS isolati nei test. Mock delle dipendenze negli unit test.
supertest	v7.0.0	Libreria per testare endpoint HTTP e integrazione.

Tabella 11: Dipendenze backend, qualità del codice

Nome	Versione	Descrizione
eslint	v9.x	Analisi statica del codice. Configurato con regole custom per complessità ciclomatica, lunghezza dei metodi e numero di parametri.
typescript-eslint	v8.x	Plugin ESLint per TypeScript.
prettier	v3.x	Formattazione automatica del codice.

Tabella 12: Dipendenze backend, integrazione cloud[G] AWS[G]

Nome	Versione	Descrizione
@aws-sdk/client-s3	v3.1017.0	Client per l'interazione con AWS S3. Utilizzato per il salvataggio dei repository scaricati e dei report generati.
@aws-sdk/client-bedrock-agent-runtime	v3.1019.0	Integrazione con Amazon Bedrock per l'esecuzione di analisi tramite Large Language Models (LLM[G]).
@aws-sdk/client-sfn	v3.1017.0	Gestione delle AWS Step Functions per il coordinamento dei workflow di analisi asincroni.

Tabella 13: Dipendenze backend, documentazione

Nome	Versione	Descrizione
@nestjs/swagger	v11.2.6	Modulo per la generazione automatica della documentazione OpenAPI tramite decorator.
swagger-ui-express	v5.0.1	Fornisce l'interfaccia grafica interattiva per esplorare e testare gli endpoint della specifica OpenAPI.

Tabella 14: Dipendenze backend, utility di analisi

Nome	Versione	Descrizione
repomix	v1.13.1	Strumento utilizzato per impacchettare l'intero repository in un formato ottimizzato per il contesto degli LLM.
adm-zip	v0.5.16	Libreria per la manipolazione di file ZIP (estrazione dei repository scaricati da sorgenti esterne).
rxjs	v7.8.1	Libreria per la programmazione reattiva tramite Observables, cuore pulsante della gestione degli eventi in NestJS.

Tabella 15: Dipendenze backend, infrastruttura serverless

Nome	Versione	Descrizione
@vendia/serverless-express	v4.12.6	Adattatore per far girare l'applicazione NestJS/Express all'interno di funzioni AWS Lambda.
serverless	v4.33.0	Framework per il deployment[G] e la gestione dell'infrastruttura Cloud come codice (IaC).

Tabella 16: Dipendenze backend, integrazione VCS[G]

Nome	Versione	Descrizione
@octokit/rest	v22.0.1	Client ufficiale per le API REST di GitHub[G]. Utilizzato per recuperare l'ultimo commit[G] SHA di un repository, verificando se l'analisi in database è aggiornata rispetto al branch[G] principale.

Tabella 17: Dipendenze backend, comunicazione HTTP

Nome	Versione	Descrizione
@nestjs/axios	v4.0.1	Modulo NestJS che integra Axios tramite Observables RxJS. Utilizzato per le chiamate HTTP verso il gateway esterno che avvia l'analisi su AWS Lambda.
axios	v1.x	Client HTTP sottostante, dipendenza di @nestjs/axios.

2.4. Runtime environment

2.4.1. NodeJS v24

Node.js è l'ambiente di runtime scelto per eseguire il codice server-side. Node.js è il runtime che consente l'esecuzione di TypeScript lato server, al di fuori del browser. Le motivazioni che hanno portato il gruppo a questa scelta progettuale sono state:

- *Uniformità del linguaggio* - L'utilizzo dello stesso linguaggio su frontend e backend elimina il context-switch, consente la condivisione di logica e tipi comuni, e semplifica la gestione delle dipendenze.
- *Architettura non bloccante e I/O asincrono* - Il modello event-driven di Node.js lo rende particolarmente adatto ad applicazioni con elevata concorrenza di richieste I/O, come chiamate a database e API esterne.
- *Ecosistema npm* - npm mette a disposizione un grande repository di librerie open source esistente.

2.5. Infrastruttura di deployment

2.5.1. Docker Engine v29.3.0

I microservizi utilizzano Docker come tecnologia di containerizzazione. Il «Dockerfile» definisce l'immagine di produzione.

2.6. Tecnologie per la persistenza dei dati

2.6.1. MongoDB v8.0.x

La scelta di MongoDB come database principale è strettamente legata alla natura della piattaforma ad agenti basata su analisi di repository. In un sistema dove i dati prodotti e consumati dagli agenti hanno strutture eterogenee, variabili nel tempo e difficilmente riducibili a uno schema relazionale fisso (come nel caso di un repository documentale piuttosto che di sviluppo ecc.), un database orientato ai documenti è la soluzione più adatta. Nello specifico:

- *Schema flessibile*: non richiede uno schema rigido predefinito. Questo è particolarmente vantaggioso in Code Guardian dove ogni agente[G] può produrre output con strutture diverse, e dove il modello dati pu; cambiare facilmente.
- *Dati gerarchici e annidati*: - I documenti JSON di MongoDB si prestano naturalmente a rappresentare strutture dati complesse e annidate (contesti agentici) senza dover ricorrere a join tra tabelle come in un database relazionale.
- *Scalabilità orizzontale* - MongoDB è progettato per scalare orizzontalmente tramite sharding nativo, caratteristica importante in una piattaforma che può dover gestire volumi crescenti di sessioni agentiche in parallelo.
- *Integrazione con l'ecosistema TypeScript* - Tramite Mongoose o il driver nativo MongoDB, la definizione di schemi e modelli tipizzati in TypeScript è diretta e ben supportata, mantenendo la coerenza con il resto della stack.

2.7. Tecnologie per infrastruttura cloud

2.7.1. AWS

AWS è il provider cloud adottato per l'intera infrastruttura della piattaforma. La scelta è motivata dalla maturità del catalogo di servizi, dall'integrazione nativa tra essi e dalla diffusione enterprise che ne garantisce un ecosistema di riferimento consolidato. In un'architettura ad agenti dove runtime serverless, orchestratori a stati, modelli LLM e storage devono comunicare in modo affidabile, operare in un unico ecosistema riduce la complessità operativa.

2.7.1.1. AWS Bedrock

Bedrock fornisce accesso a modelli fondazionali LLM tramite un'unica API gestita. Ogni agente è configurato come Bedrock Agent con un proprio system prompt e invocato dalle Lambda tramite l'SDK BedrockAgentRuntimeClient.
Il modello LLM sottostante è disaccoppiato dal codice: aggiornare versione o provider richiede solo una modifica all'alias Bedrock, senza toccare le Lambda.

2.7.1.2. AWS Lambda

Lambda è il runtime di tutta la logica agentica: pull[G] del repository, planning, esecuzione degli agenti (OWASP[G], Test, Docs[G]), aggregazione, polishing e notifica webhook sono ciascuno una funzione Lambda distinta. Il modello serverless è adatto a questo workload perché i task sono asincroni, di durata variabile e attivati su eventi; inoltre il costo è strettamente proporzionale all'elaborazione effettiva.

2.7.1.3. AWS Step Functions

Step Functions orchestra l'intero flusso agentico come macchina a stati: gestisce sequenza, parallelismo nativo (OWASP e Test Agent in contemporanea), branching condizionale sul flag runDocs, retry, timeout e propagazione degli errori verso il failure handler. Evita di dover implementare manualmente la logica di coordinamento tra Lambda e rende il flusso ispezionabile visivamente dalla console AWS.

2.7.1.4. AWS S3

S3 svolge due ruoli distinti nella piattaforma: archivia il repository clonato come ZIP che gli agenti recuperano autonomamente per l'analisi, e funge da bus di stato asincrono tra gli agenti paralleli - ogni domain agent scrive il proprio report parziale su S3 al termine dell'elaborazione, e l'orchestratore aggregatore li recupera e cancella nella fase successiva. Questo disaccoppia i componenti paralleli senza richiedere comunicazione diretta tra Lambda.
S3 viene inoltre utilizzato per l'hosting degli asset statici del frontend (MS0): il bundle prodotto da Vite in fase di build viene caricato su un bucket S3 configurato per il website hosting statico.

2.7.1.5. AWS Fargate

Fargate è il runtime serverless per container, utilizzato per hostare il frontend e il backend NestJS. Elimina la gestione dei nodi del cluster: i container vengono deployati tramite immagini ECR, scalano automaticamente e il traffico è instradato tramite Application Load Balancer. È la scelta naturale per i componenti con ciclo di vita HTTP continuo, a differenza della logica agentica che per sua natura è event-driven e gestita da Lambda.

2.8. Tecnologie per Continuous Integration[G]

2.8.1. GitHub Actions

Il progetto adotta GitHub Actions come sistema di Continuous Integration (CI[G]). Il workflow è definito nel file con estensione «.yml» e viene eseguito automaticamente ad ogni push[G] sul branch main. Il pipeline è strutturato in un job che esegue:

1. analisi statica tramite ESLint e Prettier per verificare la conformità del codice agli standard definiti, incluse metriche di qualità come complessità ciclomatica, lunghezza dei metodi e numero di parametri;
2. esecuzione degli unit test con generazione del report di coverage; esecuzione dei test di integrazione (*e2e*) con un'istanza MongoDB dedicata; verifica dell'integrità della build tramite compilazione TypeScript. Al termine del job, il report di coverage viene salvato come artifact di GitHub Actions.
3. verifica delle metriche di qualità del codice, disponibili nel documento di Piano di Qualifica v2.0.0.

3. Architettura

Il sistema è strutturato secondo un'architettura a microservizi, composta da tre componenti indipendenti che collaborano per fornire le funzionalità applicative ed una componente di frontend. Ciascun microservizio adotta internamente una Layered Architecture, suddividendo le responsabilità in strati distinti e garantendo separazione delle competenze (separation of concerns).

3.1. Architettura logica

3.1.1. Layered Architecture

Ogni microservizio è organizzato nei seguenti layer:

Presentation Layer

Costituisce il punto di ingresso del microservizio. Si occupa della ricezione delle richieste o degli eventi in ingresso, della loro validazione e della trasformazione in strutture tipizzate prima che vengano propagate agli strati sottostanti. Nel contesto frontend, questo layer corrisponde ai componenti di interfaccia utente e alla gestione degli input dell'utente.

Business Layer

Contiene la logica applicativa del microservizio, orchestrando le operazioni sui dati e coordinando le interazioni con gli altri componenti del sistema. Questo layer è esposto tramite interfacce o contratti ben definiti, in modo da disaccoppiare la logica applicativa dalla sua implementazione concreta e dai dettagli tecnologici degli strati adiacenti.

Data Layer

Gestisce l'accesso alle sorgenti dati del microservizio, siano esse layer di persistenza, API esterne o store locali. Attraverso l'uso di pattern di astrazione dedicati, garantisce che il dominio applicativo rimanga indipendente dalla tecnologia di accesso ai dati sottostante.

Infrastructure Layer (MS1)

Gestisce l'accesso alle API di servizi esterni (es. Github) o interni (es. Comunicazione fra MS1 e MS2).

3.2. Architettura di deployment

L'architettura di deployment adottata per il sistema è basata su microservizi. Questa scelta progettuale garantisce elevata scalabilità, resilienza e una totale indipendenza nello sviluppo e nel rilascio dei singoli componenti software. Ogni microservizio costituisce un'entità autonoma, responsabile[G] di un insieme specifico e circoscritto di funzionalità.

Comunicazione tra i servizi

A differenza dei sistemi basati su messaggistica asincrona, i microservizi comunicano tra loro tramite interfacce API REST (Representational State Transfer). L'adozione di questo protocollo garantisce una comunicazione chiara e ben definita tra i componenti, facilitando l'integrazione e le attività di debugging. Il modello sincrono consente inoltre un flusso di dati immediato, risultando particolarmente adatto alle operazioni agentiche che richiedono una risposta in tempo reale.

Containerizzazione e Deployment

Il deployment dei microservizi avviene in ambienti virtualizzati tramite Docker. Ogni microservizio è incapsulato in un container indipendente, operante in un ambiente isolato che previene conflitti di dipendenze e interferenze tra i servizi. Questa soluzione semplifica le fasi di test, rilascio e aggiornamento, assicurando che il software si comporti in modo identico in qualsiasi ambiente di esecuzione. La natura dei container permette infine di replicare i singoli servizi in modo rapido ed efficiente, consentendo al sistema di adattarsi dinamicamente a carichi di lavoro variabili.

3.2.1. Microservizi

Microservizio Frontend - MS0

Costituisce il punto di accesso dell'utente al sistema. Espone le seguenti funzionalità: autenticazione e visualizzazione del profilo utente, inserimento dell'URL di un repository GitHub da analizzare, e consultazione dei risultati dell'analisi. Quest'ultima comprende tre aree distinte: copertura dei test, qualità della documentazione e vulnerabilità di sicurezza secondo le linee guida OWASP. Per ciascuna area vengono presentati i suggerimenti di modifica e le motivazioni per criticità rilevate.

Microservizio di Analysis Management - MS1

Ha il compito di verificare se, per un dato repository, sia già presente in memoria un'analisi relativa all'ultimo commit disponibile. Qualora l'analisi risulti assente o non aggiornata, il microservizio provvede ad inoltrare la richiesta di analisi al microservizio competente, evitando elaborazioni ridondanti e ottimizzando l'utilizzo delle risorse computazionali.

Microservizio di Analisi dei Repository - MS2

Si occupa dell'analisi del codice sorgente delle repository mediante l'impiego di agenti software. Ricevuta una richiesta, il microservizio avvia il processo di analisi, delegando l'esecuzione a due o tre agenti specializzati e restituendo i risultati al chiamante.

Microservizio di Autenticazione e Repository Management - MS3:

Responsabile della gestione degli utenti e delle repository associate. Espone funzionalità di registrazione e autenticazione degli utenti, nonché di aggiunta e rimozione di repository. L'interazione con il servizio esterno GitHub è mediata da un componente Adapter, che isola il sistema dalle specificità dell'API esterna.

3.3. Design pattern[G]

- **Dependency Injection**

Il sistema adotta il pattern Dependency Injection tramite il container IoC di NestJS. Le dipendenze tra i layer sono mediate da interfacce, garantendo disaccoppiamento e sostituibilità delle implementazioni concrete.

- **Adapter**
  Il pattern è utilizzato per isolare i microservizi dalle specificità delle librerie esterne e dei servizi cloud.
- In MS3 (Authentication & Repository Management) e MS1 (Analysis Management), un Adapter traduce le richieste interne in chiamate conformi all'API di GitHub, permettendo al sistema di interagire con i repository senza dipendere direttamente dal formato di GitHub.
- In MS2 (Analysis Service), è stato implementato un Adapter per AWS Step Functions. Questo componente isola la logica di business di NestJS dalle specificità dell'SDK di AWS, fornendo un'interfaccia semplificata per l'avvio delle «State Machine» di analisi e gestendo internamente la conversione dei payload e degli ARN di esecuzione.
- In MS1, traduce le richieste interne in chiamate conformi all'API di GitHub, permettendo al sistema di risolvere i commit senza dipendere direttamente dal formato del provider.
- **Remote Proxy**

Il componente `AnalysisManagementInfrastructure` implementa il pattern Proxy. Esso fornisce un'interfaccia locale che rappresenta l'esecuzione di un processo remoto nel microservizio

di analisi (MS2). Il proxy gestisce internamente la complessità della comunicazione HTTP e l'autenticazione tramite API Key, rendendo l'invocazione della pipeline asincrona trasparente al Service chiamante.

- **Facade**

Il layer `AnalysisManagementPresentation` agisce come una Facade, fornendo un punto di ingresso semplificato verso l'esterno. Questo componente nasconde la complessità del workflow (che coinvolge GitHub per lo SHA, il DB per la cache e MS2 per l'analisi) dietro singoli endpoint REST, offrendo ai client un'interfaccia di alto livello per la gestione delle analisi.

- **Strategy**

Il sistema utilizza il pattern Strategy definendo interfacce per la persistenza e l'infrastruttura. Questo permette al sistema di scambiare a runtime l'implementazione effettiva (ad esempio, passando da una strategia di persistenza reale a una di test/mock) senza che il service debba conoscere i dettagli implementativi del componente iniettato.

4. Progettazione

4.1. Progettazione backend

4.1.1. Analysis Management Service - MS1

L'architettura del microservizio di gestione (MS1) è progettata per fungere da gateway e orchestratore principale del sistema. Sviluppato in NestJS, espone le API REST per l'interazione con i client, gestisce la persistenza su MongoDB e coordina le chiamate asincrone verso il microservizio di analisi (MS2).

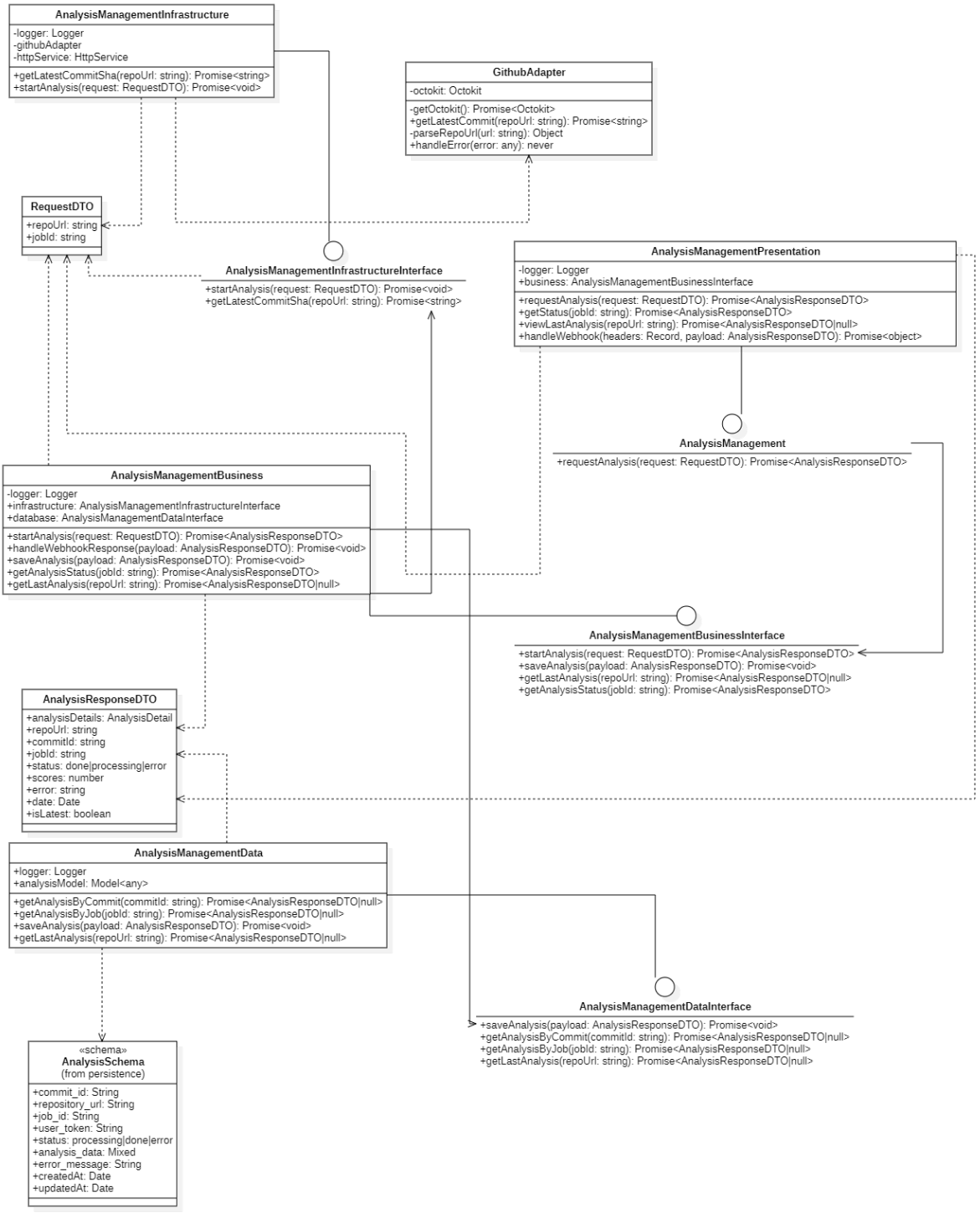

Figura 1: Diagramma delle classi; Analisi Management Service - MS1

4.1.1.1. Classi MS1 - Presentation Layer

AnalysisManagementPresentation

Punto di ingresso HTTP (NestJS). Gestisce il ciclo di vita delle richieste e riceve i feedback dai sistemi esterni.

Attributi privati:

- `analysisService: AnalysisManagementServiceInterface` - istanza del service di dominio.

Metodi pubblici:
- `requestAnalysis(payload: RequestDTO)` - Valida l'URL del repository, genera un jobId e avvia il workflow. Restituisce immediatamente un oggetto AnalysisResponseDTO con lo stato "processing".
- `getStatus(jobId: string)` - Restituisce lo stato corrente di un'analisi specifica tramite polling.
- `viewLastAnalysis(repoUrl: string)` - Recupera i dati dell'ultima analisi completata per un dato repository.
- `handleWebhook(headers, payload)` - Endpoint critico per la ricezione dei risultati da MS2. Valida la MS1_API_KEY e aggiorna il record nel database.

4.1.1.2. Classi MS1 - Business Layer

AnalysisManagementService

Service principale che implementa la logica di business e il coordinamento tra persistenza e infrastruttura.

Metodi pubblici:
- `startAnalysis(request: RequestDTO)` - Verifica se il commit è già stato analizzato (cache logic). Se nuovo, salva lo stato iniziale nel DB e invoca il trigger su MS2.
- `getAnalysisStatus(jobId: string)` - Interroga il DB per determinare se l'analisi è conclusa o ancora in corso.
- `saveAnalysis(payload: AnalysisResponseDTO)` - Conclude il processo salvando i dettagli dell'analisi (vulnerabilità, qualità, documentazione) ricevuti dal webhook.

4.1.1.3. Classi MS1 - Infrastructure Layer

AnalysisManagementInfrastructure

Gestisce le comunicazioni esterne verso GitHub e il Gateway di MS2.

Metodi pubblici:
- `getLatestCommitSha(repoUrl: string)` - Utilizza il GithubAdapter per ottenere l'hash dell'ultimo commit dal branch principale.
- `startAnalysis(request: RequestDTO)` - Esegue la chiamata HTTP POST verso l'endpoint AWS configurato (MS2_GATEWAY_URL), passando il payload necessario all'orchestrazione serverless.

GithubAdapter

Wrapper per le API di GitHub.

Metodi pubblici:
- `getLatestCommit(repoUrl: string)` - Interroga GitHub tramite Octokit per risolvere l'owner, il nome del repo[G] e l'ultimo SHA del branch di default.

4.1.1.4. Classi MS1 - Data Layer

AnalysisManagementPersistence

Modulo di interazione con MongoDB tramite Mongoose.

Metodi pubblici:
- `getAnalysisByCommit(commitId: string)` - Cerca analisi pregresse basate sull'ID univoco del commit.
- `saveAnalysis(payload: AnalysisResponseDTO)` - Esegue operazioni di upsert (update or insert) per mantenere sincronizzato lo stato dell'analisi nel database.
- `getLastAnalysis(repoUrl: string)` - Recupera l'ultimo documento inserito per un repository, ordinando per data di creazione.

4.1.1.5. MS1 - Workflow di Comunicazione

Il servizio MS1 opera come un'orchestratore a stato. Di seguito la logica di interazione con MS2:

1. Inbound: Ricezione richiesta via REST.
2. Pre-processing: Risoluzione del Commit SHA via GitHub API.
3. Persistenza: Creazione record con stato `processing`.
4. Outbound: Chiamata HTTP asincrona verso AWS Step Functions (MS2).
5. Callback: MS2 invia i risultati all'endpoint `/webhook`, portando lo stato a `done`.

4.1.2. Analisi dei Repository - MS2

L'architettura del microservizio di analisi (MS2) è progettata per gestire processi a lunga esecuzione in modo asincrono, separando la ricezione della richiesta dal workflow operativo. Il sistema si appoggia interamente ad un'infrastruttura serverless su AWS, utilizzando Step Functions per il controllo del workflow, AWS Bedrock per l'analisi tramite LLM e S3 per la persistenza dei dati e dei report intermedi.

4.1.2.1. Classi MS2 - Presentation Layer

AppController

Punto di ingresso HTTP (NestJS) per l'avvio delle pipeline di analisi. Riceve le richieste dal frontend o da MS1.

Attributi privati:

- `appService: AppService` - istanza iniettata del service per l'analisi.

Metodi pubblici:

- `startAnalysis(payload: AnalysisRequestDto)` - riceve l'URL del repository, valida il payload tramite DTO e invoca appService.triggerAnalysis(). Restituisce immediatamente una risposta contenente il messaggio di successo, un jobId e l'executionArn per il tracciamento.

AppService

Service di backend che coordina l'innesco dell'infrastruttura asincrona AWS.

Metodi pubblici:

- `triggerAnalysis(payload: AnalysisRequestDto)` - coordina l'avvio della pipeline inoltrando il payload all'adapter Step Functions tramite startStepFunctionExecution().

AwsStepFunctionsAdapter

Componente di adattamento per l'integrazione con AWS Step Functions. Isola il framework NestJS dall'utilizzo diretto dell'SDK AWS.

Attributi privati:

- `sfnClient: SFNClient` — istanza del client AWS per l'invocazione dei servizi di orchestrazione.

Metodi pubblici:

- `startStepFunctionExecution(payload: any)` — riceve il payload di analisi, inizializza un `StartExecutionCommand` e invoca la State Machine configurata tramite l'ARN presente nelle variabili d'ambiente. Restituisce l'ARN dell'esecuzione avviata.

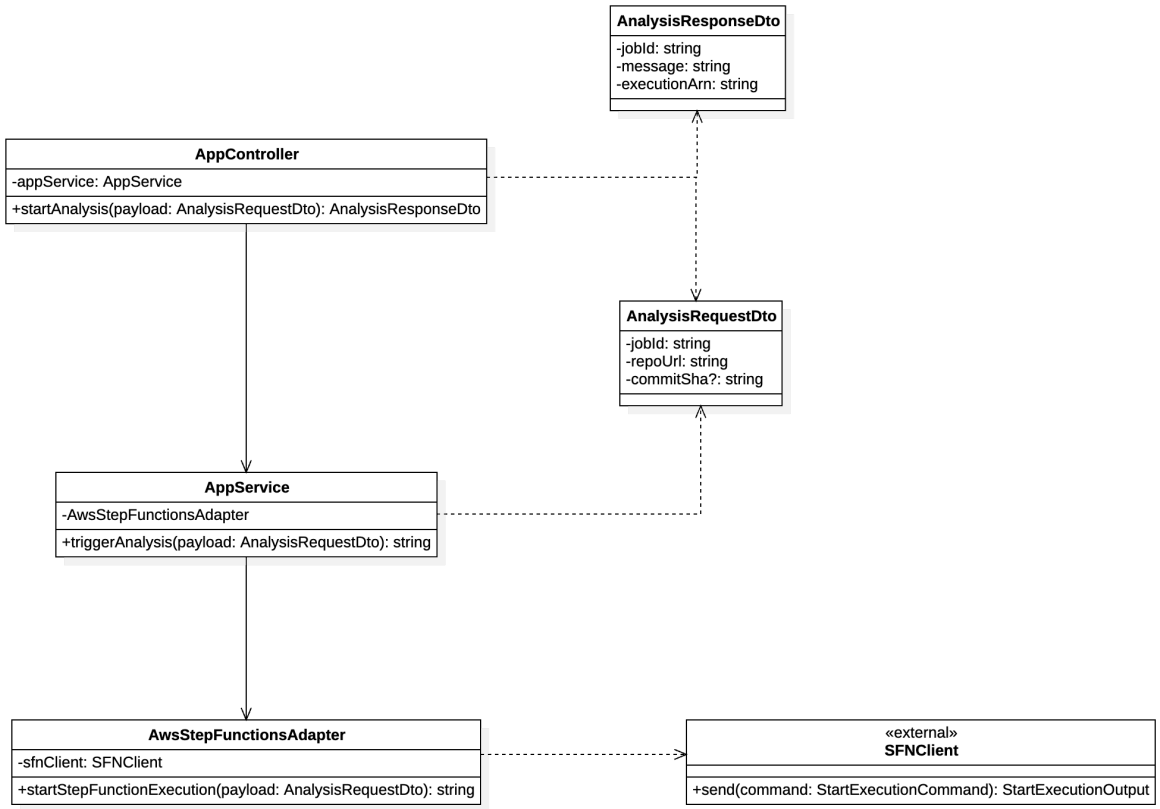

Figura 2: Presentation Layer MS2

4.1.2.2. Classi MS2 - Business Layer

4.1.2.2.1. Orchestrazione e Workflow

OrchestratorLambda

Componente centrale che gestisce il ciclo di vita dell'analisi in Step Functions, suddividendo il processo in fasi decisionali e di consolidamento.

Metodi pubblici:

- `orchestratorHandler(event: any)` - punto di ingresso Lambda che esegue le azioni principali in base al parametro action. Se action è PLAN valuta i metadati per generare il piano di esecuzione; se è AGGREGATE preleva i report da S3, aggrega i risultati per area e invoca il Master Lead Agent per il polishing finale.

4.1.2.2.2. Agenti di Analisi Tematica

OwaspAgentLambda

Funzione Lambda specializzata nell'esecuzione dell'analisi di sicurezza (OWASP) del codice sorgente.

Metodi pubblici:

- `owaspAgentHandler(event: unknown)` - scarica il file ZIP da S3 tramite unzipRepoToTemp(), utilizza SmartBundler per creare i bundle, coordina i sotto-agenti Bedrock (Dependency, Credentials, Core) e salva il report JSON finale sul bucket S3.

TestAgentLambda

Funzione Lambda specializzata nell'esecuzione dell'analisi di qualità e testing del codice sorgente.

Metodi pubblici:

- `testAgentHandler(event: unknown)` - estrae e impacchetta il codice sorgente, invoca in parallelo i sub-agenti Bedrock (QA, Boilerplate, Code Quality), consolida la sintesi e salva il report JSON su S3.

DocsAgentLambda
Funzione Lambda specializzata nell'analisi della documentazione tecnica e degli standard legali/normativi del repository.
Metodi pubblici:
- `docAgentHandler(event: unknown)` - estrae i sorgenti, delega a sub-agenti (Tech Reviewer, Compliance Officer) la revisione documentale e salva il report aggregato su S3.

4.1.2.3. Classi MS2 - Data Layer

DecompressioneZipTool
Modulo di utilità per l'interazione con AWS S3 in fase di lettura.
Metodi pubblici:
- `unzipRepoToTemp(bucket: string, zipKey: string)` — recupera l'archivio ZIP da S3 tramite GetObjectCommand, lo salva in locale e lo decomprime nel file system temporaneo.

4.1.2.4. MS2 - Workflow di Orchestrazione

Poiché le componenti del Business Layer sono implementate come funzioni Lambda serverless, prive di stato persistente e non istanziabili come oggetti, la loro struttura non si presta a una rappresentazione tramite Class Diagram UML. Il workflow di orchestrazione e le dipendenze tra queste componenti sono pertanto illustrati mediante il grafico di esecuzione della AWS Step Functions State Machine riportato di seguito.

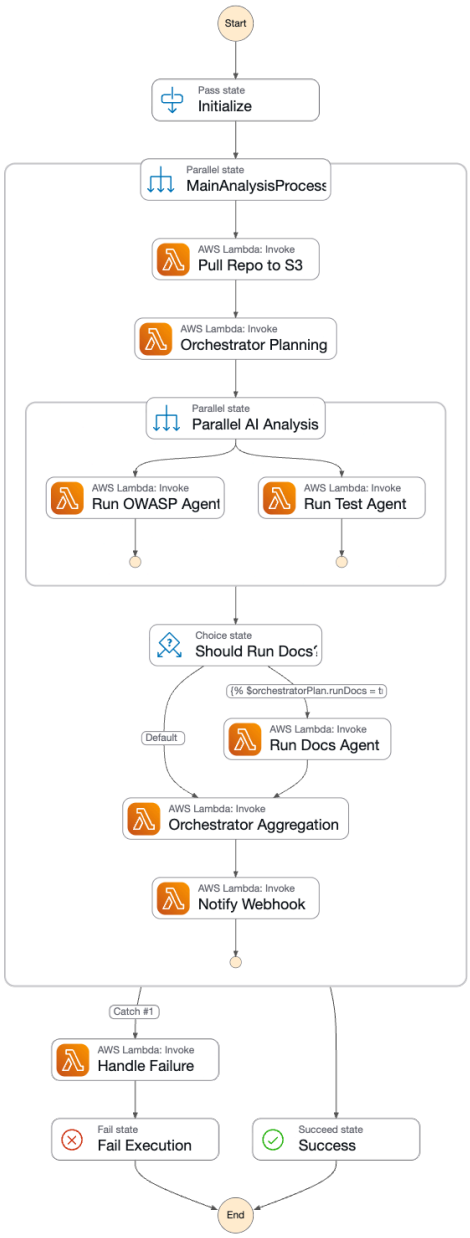

Figura 3: Workflow di orchestrazione degli agenti tramite AWS Step Functions

4.1.2.4.1. Strumenti e Utility

AgentInvoker

Modulo responsabile della comunicazione resiliente con l'infrastruttura AWS Bedrock Agents.

Metodi pubblici:

- `invokeSubAgent(agentId: string, agentAliasId: string, prompt: string, agentName: string, isLead: boolean)` - gestisce l'invocazione dell'AI. Implementa il Tool-use denial per forzare l'output testuale e include logiche di retry esponenziale per gestire il throttling.
- `extractFirstMeaningfulLine(report: string, emojiPattern: RegExp)` - estrae la sintesi dinamica dal report Markdown per il riepilogo globale.

SmartBundler

Wrapper per l'impacchettamento del codice sorgente, ottimizzato per rientrare nei limiti di contesto dei LLM tramite la libreria repomix.

Metodi pubblici:

- `createSourceChunks(extractPath: string)` — suddivide il sorgente in porzioni sequenziali (chunk) basate sulla costante `MAX_BUNDLE_CHARS` (150.000 caratteri) rispettando i confini dei file tramite analisi dei delimitatori repomix.
- `createFullChunks(extractPath: string)` — suddivide l'intero contenuto del repository in chunk per analisi complete (es. scansione credenziali).
- `extractImportedLibraries(sourceChunks: string | string[])` — esegue l'analisi statica tramite espressioni regolari per identificare le dipendenze dichiarate senza l'uso di modelli AI.

PullRepoLambda
Gestisce la fase di acquisizione iniziale del codice sorgente dal provider esterno.
Metodi pubblici:
- `handler(event: any)` - clona il repository Git[G] in una cartella temporanea, esegue il checkout sul commit specificato, estrae i metadati (tag, branch, changelog), comprime l'archivio in un file ZIP e lo carica su S3 per le fasi successive.

WebhookSenderLambda
Gestisce la notifica di completamento della pipeline.
Metodi pubblici:
- `handler(event: any)` — invia all'URL configurato un payload JSON contenente il report Markdown, «repoUrl», «jobId», «commitId» e lo stato «done». La richiesta è protetta da API Key.

FailureHandlerLambda
Gestisce le notifiche in caso di fallimento della pipeline asincrona.
Metodi pubblici:
- `handler(event: any)` - intercetta errori o crash non gestiti da AWS Step Functions e trasmette la tipologia dell'errore (errorType, cause) al sistema chiamante.

4.1.3. Autenticazione e Repository Management - MS3

Il diagramma è stato suddiviso in due figure per una questione di visibilità documentale. Il primo diagramma mostra le classi relative all'autenticazione, mentre il secondo mostra le classi relative alla gestione delle repository. L'interfaccia *IngestionInterface* e la classe *IngestionController* sono presenti in entrambi i diagrammi perché espongono funzionalità sia per l'autenticazione che per la gestione delle repository, fungendo da punto di ingresso unificato per le richieste HTTP relative a entrambe le aree funzionali.

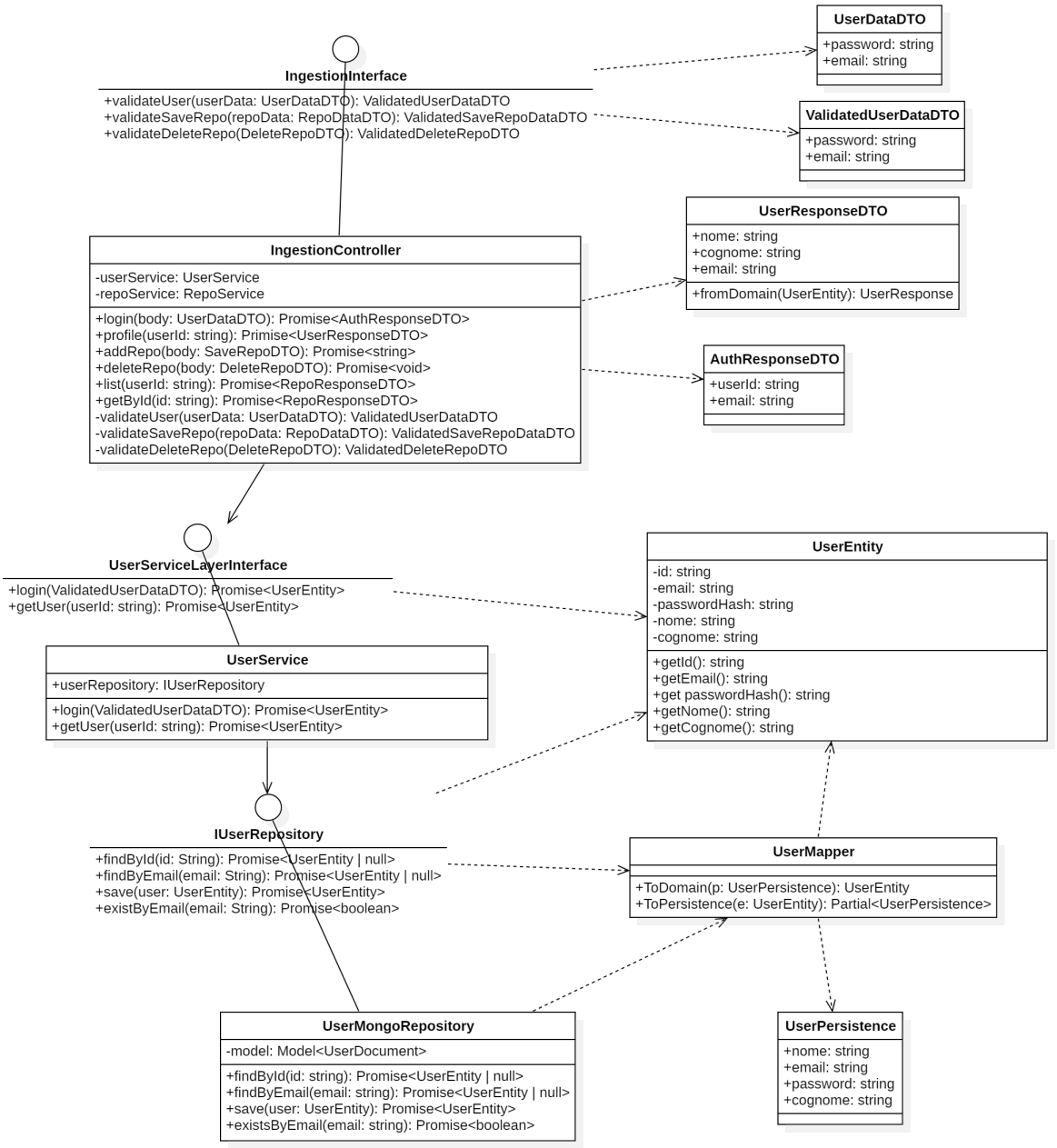

Figura 4: Diagramma delle classi; Autenticazione - MS3

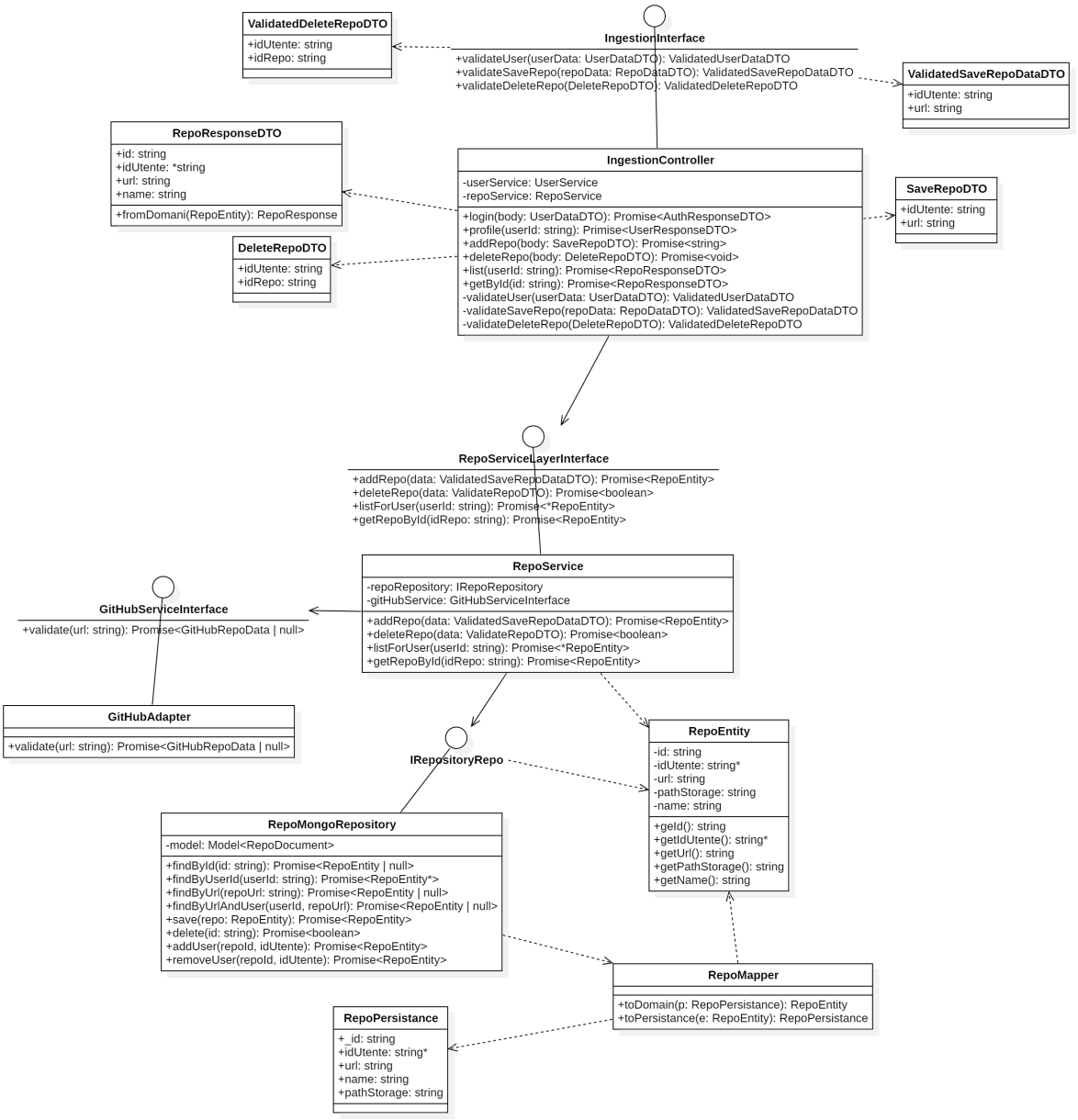

Figura 5: Diagramma delle classi; Repository Management - MS3

4.1.3.1. Classi MS3 - Presentation Layer

IngestionInterface

Definisce il contratto di validazione del controller di ingestion.

- `validateUser(data: UserDataDTO)` - contratto per la trasformazione di UserDataDTO in ValidatedUserDataDTO.
- `validateSaveRepo(data: SaveRepoDto)` - contratto per la trasformazione di SaveRepoDto in ValidatedSaveRepoDTO.
- `validateDeleteRepo(data: DeleteRepoDto)` - contratto per la trasformazione di DeleteRepoDto in ValidatedDeleteRepoDTO.

IngestionController

Punto di ingresso HTTP dell'applicazione. Riceve le richieste, le valida tramite i metodi privati e le delega ai service appropriati.

Attributi privati:

- `userService: UserService` - istanza iniettata del service utente
- `repoService: RepoService` - istanza iniettata del service repository

Metodi pubblici:
- `login(body: UserDataDTO)` - riceve le credenziali dell'utente, le valida tramite validateUser() e invoca userService.login(). Restituisce un AuthResponseDto contenente l'id e l'email dell'utente autenticato.
- `profile(userId: string)` - riceve l'id utente come query parameter, invoca userService.getUser() e restituisce un UserResponseDTO con i dati del profilo.
- `addRepo(body: SaveRepoDto)` - riceve i dati del repository, li valida tramite validateSaveRepo() e invoca repoService.addRepo(). Restituisce l'id del repository creato.
- `deleteRepo(body: DeleteRepoDto)` - riceve i dati per l'eliminazione, li valida tramite validateDeleteRepo() e invoca repoService.deleteRepo().
- `list(userId: string)` - riceve l'id utente come query parameter, invoca repoService.listForUser() e restituisce la lista dei repository associati all'utente come array di RepoResponseDto.
- `getById(id: string)` - riceve l'id del repository come query parameter, invoca repoService.getRepoById() e restituisce il repository corrispondente come RepoResponseDto.

Metodi privati:
- `validateUser(data: UserDataDTO)` - mappa un UserDataDTO in un ValidatedUserDataDTO, separando il contratto HTTP dal contratto interno dell'applicazione.
- `validateSaveRepo(data: SaveRepoDto)` - mappa un SaveRepoDto in un ValidatedSaveRepoDTO.
- `validateDeleteRepo(data: DeleteRepoDto)` - mappa un DeleteRepoDto in un ValidatedDeleteRepoDTO.

4.1.3.2. Classi MS3 - Business Layer

4.1.3.2.1. Interfacce

UserServiceLayerInterface

Definisce il contratto del service utente esposto verso il layer di presentazione.

- `login(data: ValidatedUserDataDTO)` - contratto per l'autenticazione di un utente a partire da credenziali validate.
- `getUser(userId: string)` - contratto per il recupero di un utente tramite il suo id.

RepoServiceLayerInterface

Definisce il contratto del service repository esposto verso il layer di presentazione.

- `addRepo(data: ValidatedSaveRepoDTO)` - contratto per l'aggiunta di un repository.
- `deleteRepo(data: ValidatedDeleteRepoDTO)` - contratto per l'eliminazione o la rimozione di un utente da un repository.
- `listForUser(userId: string)` - contratto per il recupero di tutti i repository associati a un utente.
- `getRepoById(idRepo: string)` - contratto per il recupero di un singolo repository tramite id.

UserRepository

Definisce il contratto di persistenza per l'entità utente, implementato a livello infrastrutturale.

- `findById(id: string)` - recupera un utente tramite il suo id. Restituisce null se non trovato.

- `findByEmail(email: string)` - recupera un utente tramite email. Restituisce null se non trovato.
- `save(user: UserEntity)` - persiste un nuovo utente e restituisce l'entità salvata.
- `existsByEmail(email: string)` - verifica se esiste già un utente con la email fornita. Restituisce un booleano.

RepoRepository
Definisce il contratto di persistenza per l'entità repository, implementato a livello infrastrutturale.

- `findById(id: string)` - recupera un repository tramite il suo id. Restituisce null se non trovato.
- `findByUserId(userId: string)` - recupera tutti i repository associati a un determinato utente.
- `findByUrl(repoUrl: string)` - recupera un repository tramite URL. Restituisce null se non trovato. Utile per verificare se un repository è già presente nel sistema prima di salvarlo.
- `findByUrlAndUser(userId, repoUrl)` - recupera un repository tramite URL filtrandolo per utente specifico. Restituisce null se non trovato.
- `save(repo: RepoEntity)` - persiste un nuovo repository e restituisce l'entità salvata.
- `delete(id: string)` - elimina definitivamente un repository dal sistema. Restituisce true se l'operazione è andata a buon fine.
- `addUser(repoId, idUtente)` - aggiunge un utente all'array idUtente del repository specificato. Usato quando un repository esiste già e un nuovo utente vuole aggiungerlo. Restituisce il repository aggiornato.
- `removeUser(repoId, idUtente)` - rimuove un utente dall'array idUtente del repository specificato. Usato quando un utente elimina un repository che è condiviso con altri. Restituisce il repository aggiornato.

GitHubServiceInterface
Definisce il contratto per la validazione di un repository GitHub tramite API esterna.

- `validate(url: string)` - verifica che l'URL fornito corrisponda a un repository GitHub pubblico e accessibile. Restituisce un oggetto GitHubRepoData con i metadati del repository, oppure null se il repository non esiste o non è accessibile.

4.1.3.2.2. Servizi

UserService
Implementa UserServiceLayerInterface. Contiene la logica applicativa relativa agli utenti.
Attributi privati:
- `userRepository: IUserRepository` - istanza iniettata del repository utente.

Metodi pubblici:
- `login(data: ValidatedUserDataDTO)` - recupera l'utente tramite email. Se non trovato lancia UnauthorizedException. Verifica la correttezza della password tramite bcrypt.compare(). Se la verifica fallisce lancia UnauthorizedException. Restituisce la UserEntity dell'utente autenticato.
- `getUser(userId: string)` - recupera l'utente tramite id. Se non trovato lancia NotFoundException. Restituisce la UserEntity.

RepoService
Implementa RepoServiceLayerInterface. Contiene la logica applicativa relativa ai repository.
Attributi privati:
- `repoRepository: IRepoRepository` - istanza iniettata del repository.
- `githubService: GitHubServiceInterface` - istanza iniettata dell'adapter GitHub.

Metodi pubblici:
- `addRepo(data: ValidatedSaveRepoDTO)` - valida il repository tramite githubService.validate(). Se non valido lancia BadRequestException. Verifica se il repository esiste già tramite findByUrl(). Se esiste e l'utente è già associato lancia ConflictException. Se esiste ma l'utente non è associato, aggiunge l'utente tramite addUser(). Se non esiste, crea una nuova RepoEntity con un UUID generato e la persiste tramite save(). Restituisce la RepoEntity.
- `deleteRepo(data: ValidatedDeleteRepoDTO)` - recupera il repository tramite id. Se non trovato lancia NotFoundException. Se l'utente non è tra quelli associati lancia NotFoundException. Se il repository ha più utenti associati, rimuove solo l'utente corrente tramite removeUser(). Se l'utente è l'unico, elimina definitivamente il repository tramite delete(). Restituisce un booleano.
- `listForUser(userId: string)` - recupera tutti i repository associati all'utente tramite findByUserId(). Restituisce un array di RepoEntity.
- `getRepoById(idRepo: string)` - recupera il repository tramite id. Se non trovato lancia NotFoundException. Restituisce la RepoEntity.

4.1.3.2.3. Entità di dominio

UserEntity

Rappresenta l'entità di dominio dell'utente. Tutti gli attributi sono readonly e accessibili solo tramite getter, garantendo l'immutabilità.

Attributi privati:
- `id: string, nome: string, cognome: string, email: string, passwordHash: string`

Getter pubblici:
- id, nome, cognome, email, passwordHash

RepoEntity

Rappresenta l'entità di dominio del repository. Tutti gli attributi sono readonly e accessibili solo tramite getter. L'attributo idUtente è un array per supportare la condivisione di un repository tra più utenti.

Attributi privati:
- `id: string, idUtente: string*, url: string, name: string, pathStorage: string`

Getter pubblici:
- id, idUtente, url, name, pathStorage

4.1.3.3. Classi MS3 - Data Layer

UserPersistence

Schema Mongoose per la collezione users. Mappa la struttura di UserEntity al documento MongoDB.

Attributi pubblici:
- `nome, cognome, email, passwordHash` - quest'ultimo con select: false per escluderlo di default dalle query e includerlo solo quando esplicitamente necessario.

RepoPersistence

Schema Mongoose per la collezione repos. Mappa la struttura di RepoEntity al documento MongoDB.

Attributi pubblici:
- `_id, idUtente (array), url, name, pathStorage`

UserMapper

Classe di conversione tra UserEntity e UserPersistence. Tutti i metodi sono statici.

- `toDomain(p: UserPersistence)` - converte un documento Mongoose in una UserEntity istanziando la classe con i campi corrispondenti.
- `toPersistence(e: UserEntity)` - converte una UserEntity in un oggetto parziale Partial<UserPersistence> con i campi necessari alla persistenza. Non include l'id in quanto gestito da MongoDB.

RepoMapper
Classe di conversione tra RepoEntity e RepoPersistence. Tutti i metodi sono statici.

- `toDomain(p: RepoPersistence)` - converte un documento Mongoose in una RepoEntity istanziando la classe con i campi corrispondenti.
- `toPersistence(e: RepoEntity)` - converte una RepoEntity in un oggetto RepoPersistence con tutti i campi necessari alla persistenza, incluso _id.

UserMongoRepository
Implementa IUserRepository. Gestisce le operazioni di persistenza degli utenti su MongoDB tramite Mongoose.
Attributi privati:
- `model: Model<UserDocument>` - modello Mongoose iniettato tramite @InjectModel.

Metodi pubblici:
- `findById(id: string)` - esegue una query findById aggiungendo .select(«+passwordHash») per includere il campo normalmente escluso. Converte il documento tramite UserMapper.toDomain(). Restituisce null se non trovato.
- `findByEmail(email: string)` - esegue una query findOne per email aggiungendo .select(«+passwordHash»). Converte il documento tramite UserMapper.toDomain(). Restituisce null se non trovato.
- `save(user: UserEntity)` - converte l'entità tramite UserMapper.toPersistence() e la persiste tramite model.create(). Restituisce l'entità convertita tramite UserMapper.toDomain().
- `existsByEmail(email: string)` - esegue una countDocuments per email e restituisce true se il conteggio è maggiore di zero.

RepoMongoRepository
Implementa IRepoRepository. Gestisce le operazioni di persistenza dei repository su MongoDB tramite Mongoose.
Attributi privati:
- `model: Model<RepoDocument>` - modello Mongoose iniettato tramite @InjectModel.

Metodi pubblici:
- `findById(id: string)` - esegue findById e converte il documento tramite RepoMapper.toDomain(). Restituisce null se non trovato.
- `findByUserId(userId: string)` - esegue find filtrando per idUtente, ordinando per data di creazione decrescente con limite di 100 risultati. Converte ogni documento tramite RepoMapper.toDomain().
- `findByUrl(repoUrl: string)` - esegue findOne filtrando per url. Restituisce null se non trovato.
- `findByUrlAndUser(userId, repoUrl)` - esegue findOne filtrando per url e idUtente contemporaneamente. Restituisce null se non trovato.
- `save(repo: RepoEntity)` - converte l'entità tramite RepoMapper.toPersistence() e la persiste tramite model.create(). Restituisce l'entità convertita.
- `delete(id: string)` - esegue findByIdAndDelete. Restituisce true se il documento è stato trovato ed eliminato, false altrimenti.

- `addUser(repoId, idUtente)` - esegue findByIdAndUpdate con operatore $push per aggiungere l'utente all'array. Se il documento non esiste lancia NotFoundException. Restituisce il repository aggiornato convertito tramite RepoMapper.toDomain().
- `removeUser(repoId, idUtente)` - esegue findByIdAndUpdate con operatore $pull per rimuovere l'utente dall'array. Se il documento non esiste lancia NotFoundException. Restituisce il repository aggiornato convertito tramite RepoMapper.toDomain().

GitHubAdapter
Implementa GitHubServiceInterface. Gestisce la comunicazione con le API pubbliche di GitHub.

- `validate(url: string)` - interpreta l'URL GitHub fornito, estrae owner e nome del repository ed esegue una chiamata alle API di GitHub per verificarne l'esistenza e l'accessibilità pubblica. Restituisce un oggetto GitHubRepoData con i metadati del repository se la chiamata ha successo, null altrimenti.

4.2. Progettazione frontend

Il frontend è sviluppato in React con TypeScript e Vite. L'architettura è organizzata in tre layer: Presentation, Business e Data. Il layer di presentazione comprende il sistema di routing, le pagine e i componenti riutilizzabili. Il layer di business raccoglie i service che incapsulano la logica applicativa e le chiamate HTTP. Il layer data contiene la configurazione degli endpoint e le interfacce TypeScript che definiscono il contratto con il backend.

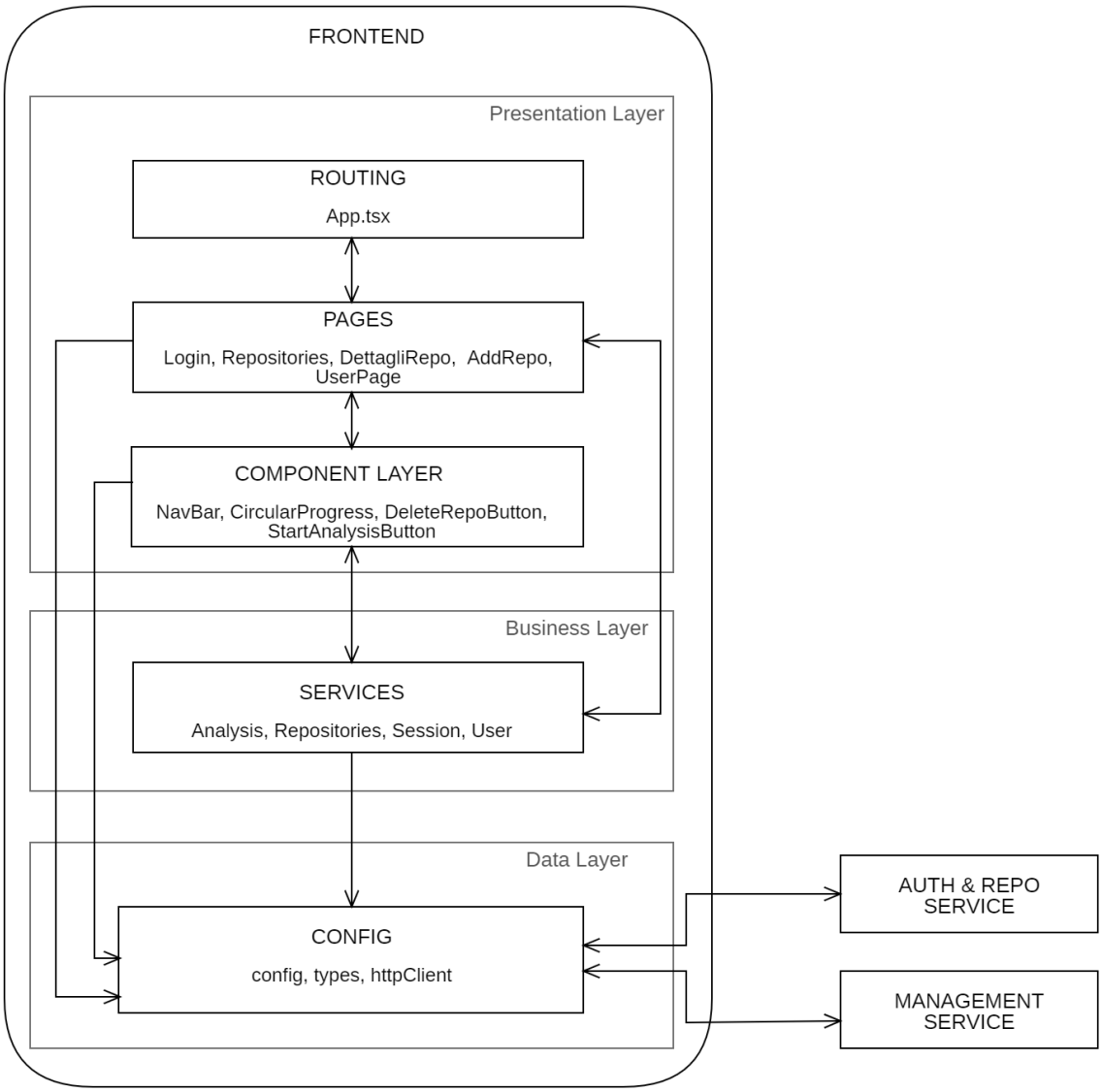

Figura 6: Layered Architecture - frontend

4.2.1. Moduli MS0 - Presentation Layer

4.2.1.1. Routing

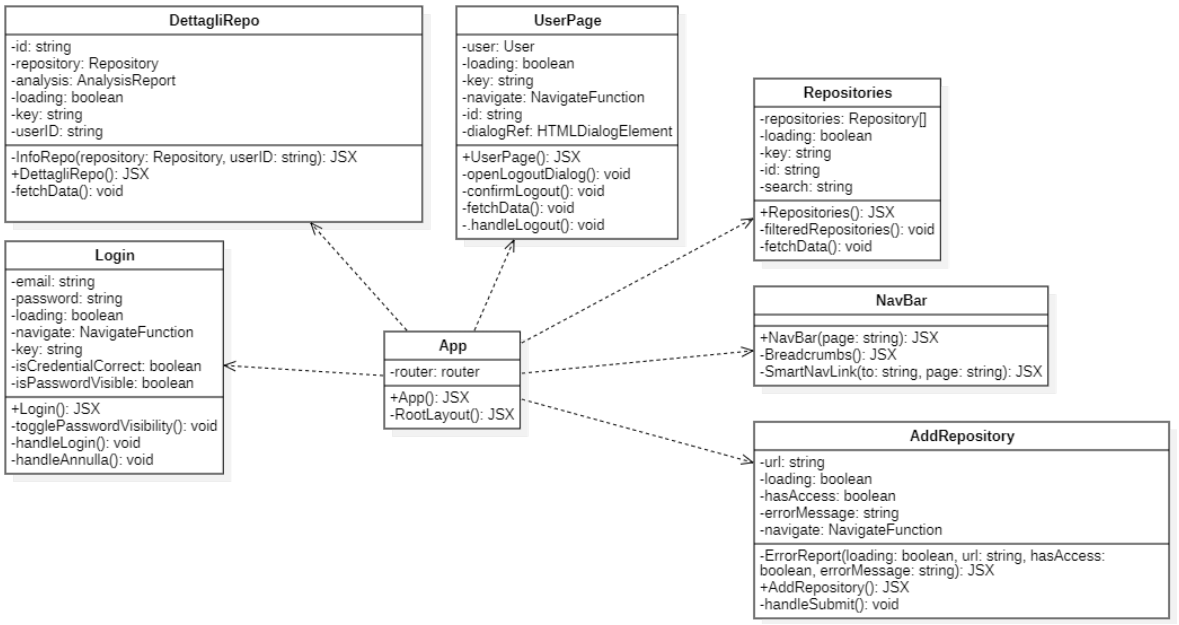

Figura 7: Diagramma componenti, App - frontend

App
Punto di ingresso dell'applicazione. Configura il router tramite `createBrowserRouter` e definisce la struttura delle route. La route radice / utilizza RootLayout come elemento padre e reindirizza automaticamente a /repositories. Le route figlie sono /repositories, /repository/:id, /addRepository e /profile. La route /login è definita separatamente, al di fuori di RootLayout, in quanto non prevede la barra di navigazione.

- `RootLayout` - Componente strutturale che avvolge tutte le pagine protette. Renderizza NavBar in cima e Outlet come contenitore per le pagine figlie iniettate dal router. Non contiene logica applicativa.

4.2.1.2. Componenti

NavBar
Barra di navigazione persistente presente in tutte le pagine protette. Composta internamente da Breadcrumbs e SmartNavLink.

- `Breadcrumbs` - Sottocomponente interno di NavBar. Legge i match correnti tramite useMatches e costruisce dinamicamente una lista di link basandosi sull'attributo *handle.label* definito nelle route. L'ultimo elemento della lista è renderizzato come testo non cliccabile.
- `SmartNavLink` - Sottocomponente interno di NavBar. Riceve *to* e *page* come props. Tramite useLocation confronta il pathname corrente con la destinazione: se coincidono renderizza un paragrafo statico invece di un link, evitando la navigazione ridondante verso la pagina già attiva.

CircularProgress
Componente di visualizzazione per valori percentuali. Riceve *percentage*, *label* e *size* opzionale come props. La classe CSS del cerchio di progresso varia in base alla soglia della percentuale tramite la funzione interna `getColorClass` : top-performance per valori ≥ 90, medium-performance per valori ≥ 50, bad-performance altrimenti.

DeleteRepoButton
Componente per l'eliminazione di un repository. Riceve *repository*, *userID*, *messageButton* opzionale e *onDeleted* opzionale come props. Gestisce due nativi tramite ref: uno di conferma prima dell'eliminazione e uno di errore in caso di fallimento. Al conferma invoca deleteRepo() del RepositoriesService. Se *onDeleted* è definito lo richiama per aggiornare lo stato del componente padre; altrimenti naviga a /repositories.

StartAnalysisButton
Componente per l'avvio e il monitoraggio di un'analisi. Riceve *url*, *messageButton* opzionale, *onSuccess* opzionale e *initialJobId* opzionale come props. Se *initialJobId* è presente, avvia il polling immediatamente tramite useEffect per riprendere un'analisi già in corso. Gestisce due nativi: uno di conferma prima dell'avvio e uno di errore. Il pulsante è disabilitato durante il polling con testo "Analisi in corso...".

Metodi privati:

- `startPolling(jobId: string)` - avvia un *setInterval* con cadenza di 3000ms. Ad ogni tick invoca *pollAnalysisStatus()*. Se lo status è «done» chiude l'intervallo e richiama onSuccess. Se è «error» chiude l'intervallo e mostra il dialog di errore. Se supera il timeout di 15 minuti chiude l'intervallo e mostra il messaggio di timeout.
- `handleConfirm()` - chiude il dialog di conferma e invoca *startNewAnalysis()*. Se la risposta ha status «done» richiama direttamente onSuccess senza polling. Se ha status «processing» avvia il polling con il jobId ricevuto.

4.2.1.3. Pagine

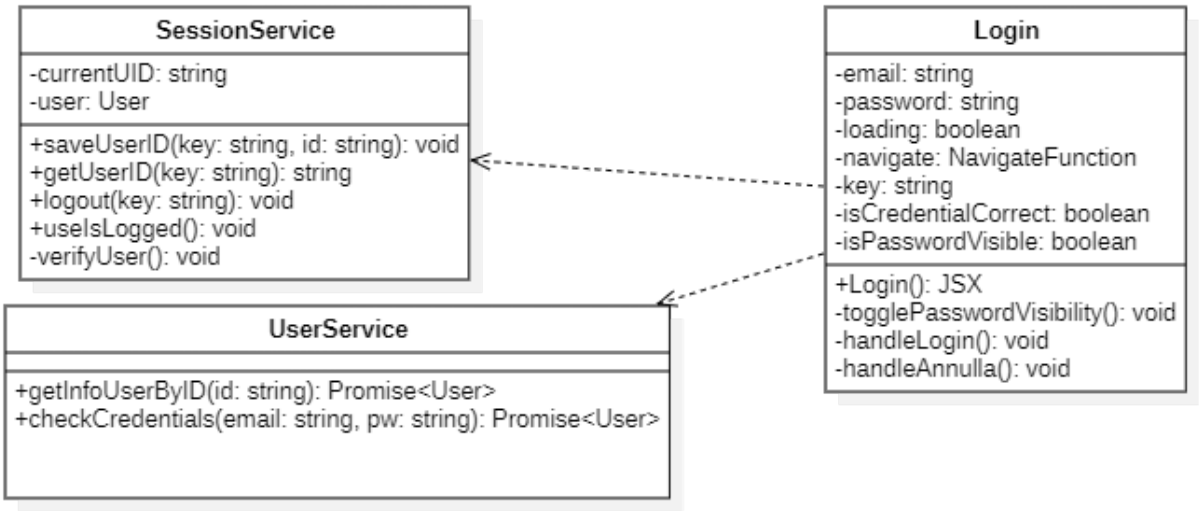

Figura 8: Diagramma componenti, Login - frontend

Login
Pagina di autenticazione, esterna al RootLayout. Invoca *useIsLogged()* per reindirizzare automaticamente l'utente già autenticato. Gestisce lo stato locale per email, password, isPasswordVisible, isCredentialCorrect e loading. Al submit invoca *checkCredentials()* del *UserService*, salva lo *userId* tramite *saveUserID()* del *SessionService* e naviga a /repositories. In caso di errore imposta *isCredentialCorrect* a false e mostra il messaggio di errore.

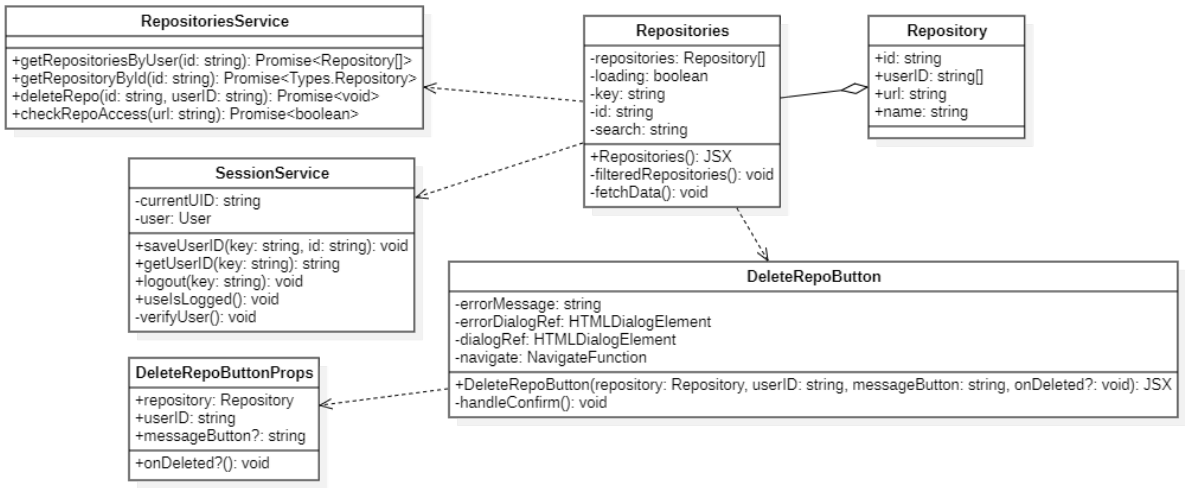

Figura 9: Diagramma componenti, Repositories- frontend

Repositories

Pagina principale dell'applicazione. Invoca *useIsLogged()* e recupera lo *userId* dalla sessione tramite *getUserID()*. All'inizio invoca *getRepositoriesByUser()* e popola lo stato repositories. Filtra la lista in tempo reale tramite useMemo in base al valore della search bar. Renderizza ogni repository come elemento di lista con un link a /repository/:id e un *DeleteRepoButton*. Alla cancellazione riuscita, aggiorna lo stato locale rimuovendo l'elemento senza ricaricare la pagina.

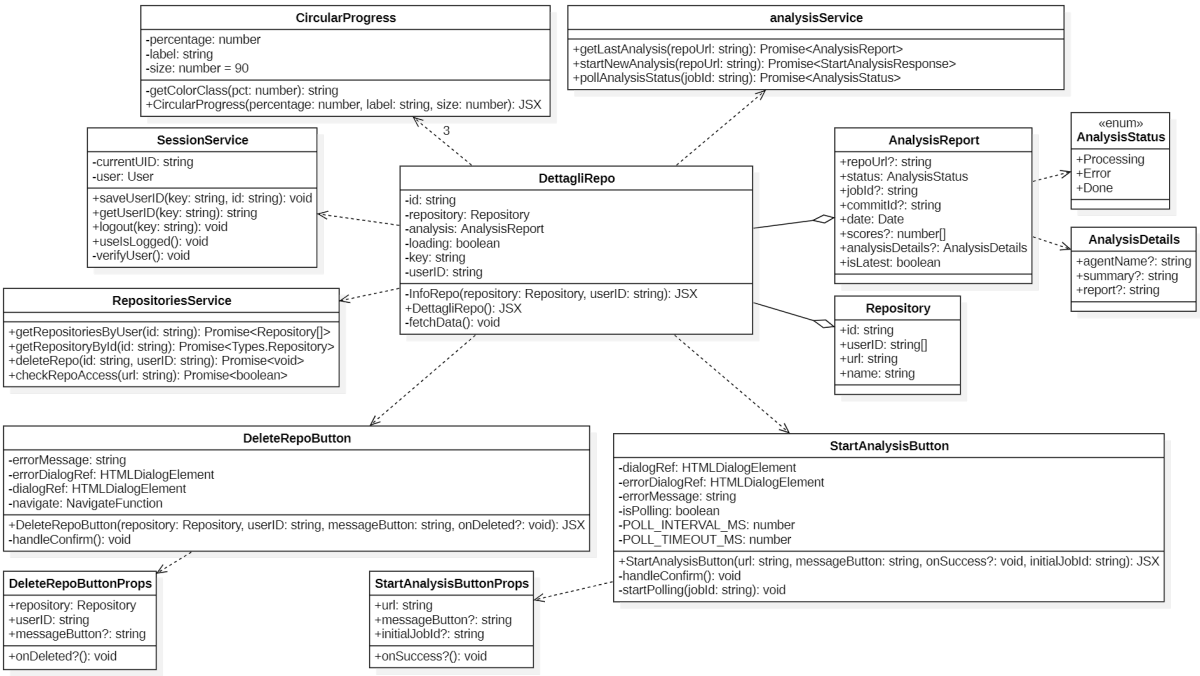

Figura 10: Diagramma componenti, DettagliRepo- frontend

DettagliRepo

Pagina di dettaglio di un repository. Legge l'id dai parametri URL tramite useParams. All'inizio invoca in sequenza *getRepositoryById()* e *getLastAnalysis()*. Espone la funzione fetchData passata come *onSuccess* a *StartAnalysisButton*, così al termine del polling la pagina si ricarica automaticamente. Se *analysis.status* è «processing» passa il *commitId* come *initialJobId* a *StartAnalysisButton* per riprendere il polling. Composta internamente da InfoRepo.

Sottocomponente interno:

- `InfoRepo` - aside che riceve *repository* e *userID* come props. Renderizza il nome del repository, un *DeleteRepoButton* con label «Elimina repository», un link esterno alla pagina GitHub e un link di navigazione a /repositories.

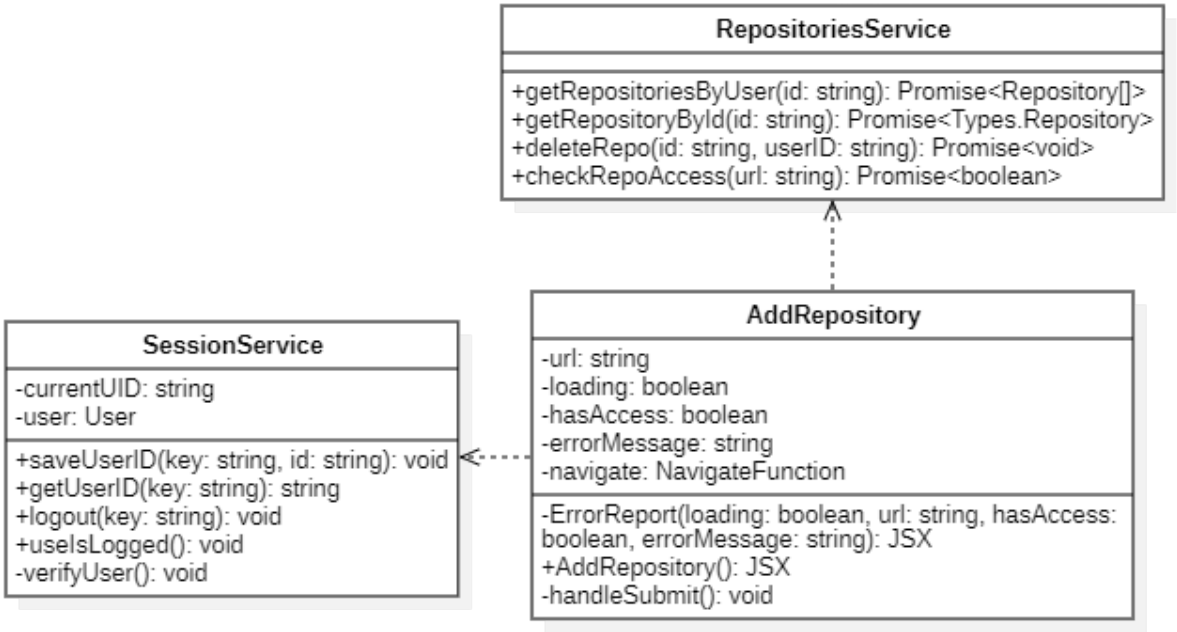

Figura 11: Diagramma componenti, AddRepository- frontend

AddRepository

Pagina per l'aggiunta di un nuovo repository. Invoca *useIsLogged()*. Gestisce lo stato locale per url, loading, *hasAccess* e errorMessage. Al submit invoca *checkRepoAccess()* del *RepositoriesService*. In caso di successo naviga a /repositories. In caso di errore imposta *hasAccess* a false e mostra il messaggio di errore proveniente dal backend. Composta internamente da ErrorReport.

Sottocomponente interno:

- `ErrorReport` - riceve loading, url, *hasAccess* ed errorMessage come props. Renderizza il messaggio di caricamento se loading è true, il messaggio di errore se *hasAccess* è false, nulla altrimenti.

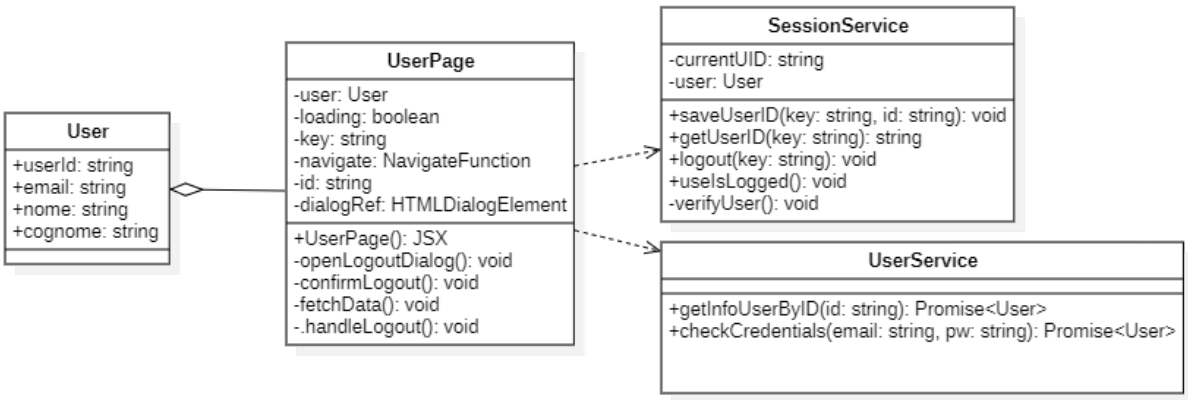

Figura 12: Diagramma componenti, UserPage - frontend

UserPage

Pagina del profilo utente. Invoca *useIsLogged()* e recupera lo userId dalla sessione. All'inizio invoca *getInfoUserByID()* del *UserService*. Gestisce il caso in cui l'utente sia presente in localStorage ma non

nel database: in tal caso mostra solo il pulsante di logout, senza un di conferma, il quale è invece presente nel normale logout. Alla conferma del logout da , viene invocato *logout()* da *SessionService* e naviga a /login.

4.2.2. Moduli MS0 - Business Layer

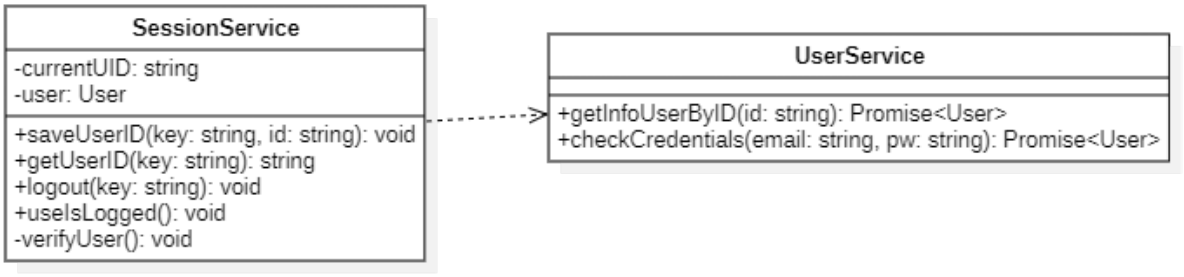

Figura 13: Diagramma componenti, SessionService - frontend

SessionService
Gestisce la sessione utente tramite localStorage e la guardia di navigazione. Esporta le seguenti funzioni:

- `saveUserID(key: string, value: string)` - salvataggio dell'identificativo utente in localStorage. invoca *localStorage.setItem()* con la chiave e il valore forniti.
- `getUserID(key: string)` - recupero dell'identificativo utente da localStorage. invoca *localStorage.getItem()* e restituisce il valore associato alla chiave, oppure null se assente.
- `logout(key: string)` - rimozione dell'identificativo utente da localStorage. invoca *localStorage.removeItem()* per eliminare la chiave fornita.
- `useIsLogged()` - React hook che tramite useEffect verifica la presenza dello userId in localStorage e il percorso corrente. Se l'utente non è autenticato e non si trova in /login, reindirizza a /login. Se è autenticato e si trova in /login, reindirizza a /repositories. Se è autenticato ma non trovato nel database tramite getInfoUserByID(), reindirizza a /profile salvo che non ci si trovi già lì.

Nota: la persistenza dell'identificativo utente tramite localStorage è stata scelta per semplicità implementativa nel contesto del progetto didattico. Si è consapevoli che in un contesto produttivo questa soluzione sarebbe sostituita da soluzioni più sicure. La scelta attuale è considerata accettabile dato il perimetro del progetto.

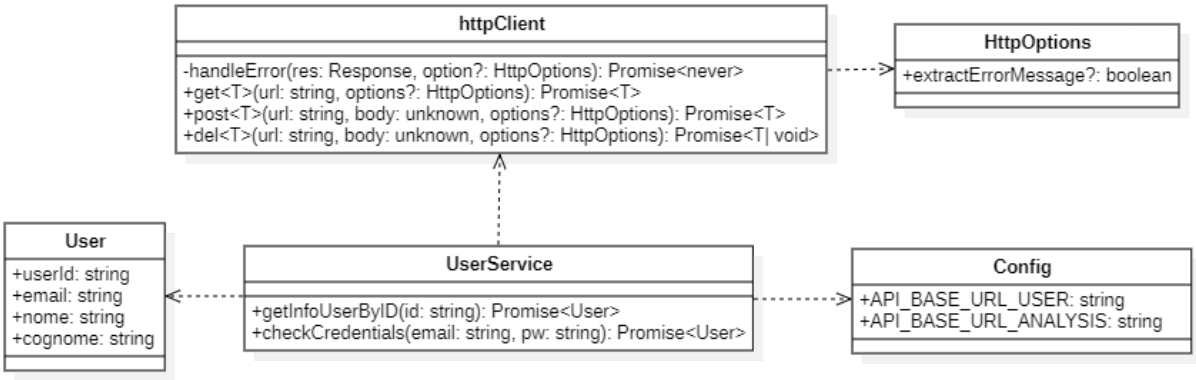

Figura 14: Diagramma componenti, UserService - frontend

UserService
Gestisce l'integrazione con lo userService del backend, esponendo al layer di presentazione le operazioni necessarie per la gestione dell'utente. Esporta le seguenti funzioni:

- `getInfoUserByID(id: string)` - recupero dei dati di un utente tramite id. invoca *get<User>()* sull'endpoint /profile?userId={id}. Restituisce una User oppure lancia errore se non trovato.
- `checkCredentials(email: string, password: string)` - verifica delle credenziali di accesso. Invoca *post<User>()* sull'endpoint /auth/login con body { email, password }. Restituisce la User autenticata oppure lancia errore se le credenziali non sono valide.

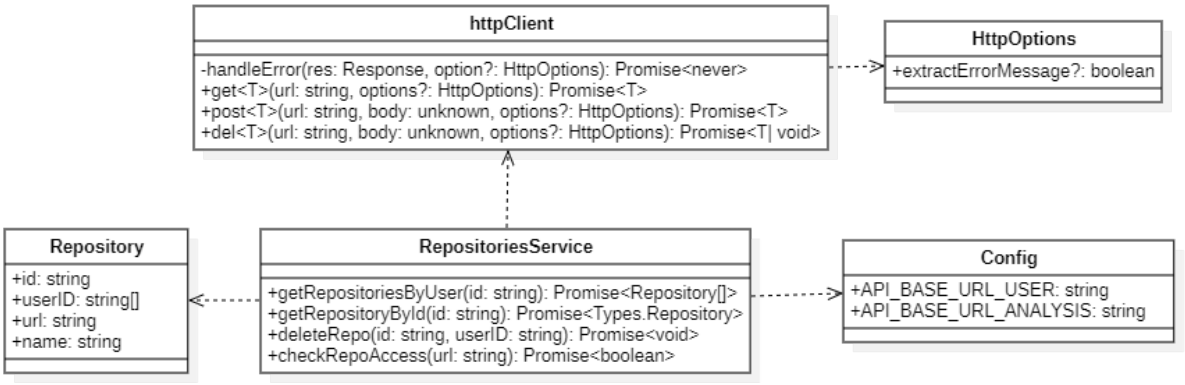

Figura 15: Diagramma componenti, RepositoriesService - frontend

RepositoriesService
Gestisce l'integrazione con lo userService del backend, esponendo al layer di presentazione le operazioni necessarie per la gestione e la consultazione dei repository. Esporta le seguenti funzioni:

- `getRepositoriesByUser(id: string)` - recupero dei repository associati a un utente. Invoca *get<Repository[]>()* sull'endpoint /repos?userId={id}. Restituisce un array di Repository.
- `getRepositoryById(id: string)` - recupero di un singolo repository tramite id. Invoca *get<Repository>()* sull'endpoint /repo?repoId={id}. Restituisce la Repository oppure lancia errore se non trovata.
- `deleteRepo(idRepo: string, idUtente: string)` - eliminazione di un repository. Invoca *del()* sull'endpoint /repo con body { idUtente, idRepo }. Restituisce true se l'eliminazione va a buon fine.
- `checkRepoAccess(url: string)` - verifica dell'accessibilità di un repository GitHub per la sua aggiunta al sistema. Recupera lo userId tramite *getUserID()* del *SessionService* e invoca *post()* sull'endpoint /repo con body { idUtente, url } e opzione { extractErrorMessage: true }. Restituisce true se il repository è accessibile. In caso di errore propaga il messaggio ricevuto dal backend.

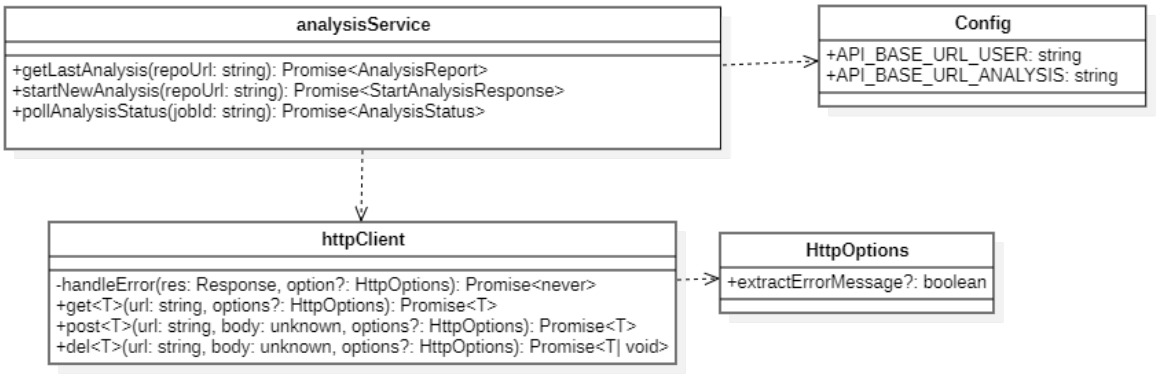

Figura 16: Diagramma componenti, AnalysisService - frontend

AnalysisService
Gestisce l'integrazione con il managementService del backend, esponendo al layer di presentazione le funzionalità necessarie per la gestione e il coordinamento del sistema. Esporta le seguenti funzioni:

- `getLastAnalysis(repoUrl: string)` - contratto per il recupero dell'ultima analisi disponibile per un repository. Invoca *get<AnalysisReport>()* sull'endpoint /analysis/view?url={repoUrl}. Restituisce un AnalysisReport oppure lancia errore se non trovato.
- `startNewAnalysis(repoUrl: string)` - contratto per l'avvio di una nuova analisi. Invoca *post<StartAnalysisResponse>()* sull'endpoint /analysis/request con body { repoUrl }. Restituisce uno StartAnalysisResponse con status: «done» se l'analisi è già disponibile, oppure status: «processing» con un jobId da usare per il polling.
- `pollAnalysisStatus(jobId: string)` - contratto per il controllo dello stato di un'analisi in corso. Invoca *get<{ status: AnalysisStatus }>()* sull'endpoint /analysis/status/{jobId} e restituisce il solo campo status.

4.2.3. Moduli MS0 - Data Layer

4.2.3.1. HTTP Client

httpClient

- `HttpOptions` - Interfaccia di configurazione per le richieste HTTP.
- `extractErrorMessage?: boolean` - Se true, il client tenta di estrarre il messaggio di errore dal body JSON della risposta invece di usare statusText. Utilizzato selettivamente per gli endpoint che restituiscono messaggi significativi nel body.
- `httpClient` - Modulo che espone le funzioni HTTP di base utilizzate dai service. Tutte le funzioni delegano la gestione degli errori a handleError.

Funzioni interne:

- `handleError(res: Response, options?: HttpOptions)` - Se *extractErrorMessage* è attivo, tenta di deserializzare il body JSON tramite *res.json().catch(() => null)*. Se il campo message è un array contenente una stringa con «url must be», lo rimappa in «URL non valido.». Se è un array senza quel pattern, usa il primo elemento. Se è una stringa, la usa direttamente. In tutti i casi di fallback usa res.statusText. Se *extractErrorMessage* non è attivo, lancia direttamente *new Error(res.statusText)*.

Funzioni esportate:

- `get<T>(url: string, options?: HttpOptions)` - esegue una fetch con metodo GET. Se !res.ok delega a handleError. Altrimenti deserializza e restituisce il tipo T.
- `post<T>(url: string, body: unknown, options?: HttpOptions)` - esegue una fetch con metodo POST, header Content-Type: application/json e body serializzato. Se !res.ok delega a handleError. Altrimenti restituisce il tipo T.
- `del<T>(url: string, body: unknown, options?: HttpOptions)` - esegue una fetch con metodo DELETE, header Content-Type: application/json e body serializzato. Se !res.ok delega a handleError. Altrimenti restituisce il tipo T.

4.2.3.2. Tipi di dominio

User
Interfaccia TypeScript che rappresenta l'entità utente ricevuta dal backend.
Attributi: userId: string, nome: string, cognome: string, email: string.

Repository
Interfaccia TypeScript che rappresenta l'entità repository ricevuta dal backend.
Attributi: id: string, userID: string[], url: string, name: string.

AnalysisDetails
Interfaccia TypeScript che rappresenta il dettaglio di una singola analisi eseguita da un agente.
Attributi: agentName?: string, summary?: string, report?: string.

AnalysisReport
Interfaccia TypeScript che rappresenta il report di un'analisi ricevuto dal backend.
Attributi: repoUrl?: string, jobId?: string, commitId?: string, status: AnalysisStatus, analysisDetails?: AnalysisDetails[], scores?: number[], date: Date, isLatest:boolean.

AnalysisStatus
Tipo unione che rappresenta i possibili stati di un'analisi: «done» | «processing» | «error».

StartAnalysisResponse
Interfaccia TypeScript che rappresenta la risposta all'avvio di una nuova analisi.
Attributi: status: AnalysisStatus, jobId?: string, commitId?: string, repoUrl: string.

4.2.3.3. Configurazione

config
Modulo che espone le costanti di configurazione degli URL base delle API, lette dalle variabili d'ambiente Vite al build time.

- `API_BASE_URL_USER` — URL base del microservizio utenti, letto da VITE_URL_USER_SERVICE per le verifiche in localhost. Fallback al link del repository su AWS se la variabile non è definita.
- `API_BASE_URL_ANALYSIS` — URL base del microservizio analisi, letto da VITE_URL_ANALYSIS per le verifiche in localhost. Fallback al link del repository su AWS se la variabile non è definita.

5. Stato dei requisiti funzionali

5.1. Stato per requisito

Tabella 18: Copertura requisiti funzionali

Codice	Descrizione	Stato
R-1-F-O	L'Utente Sconosciuto deve poter accedere alla piattaforma	Soddisfatto
R-2-F-O	L'Utente Sconosciuto deve poter inserire il proprio username	Soddisfatto
R-3-F-O	L'Utente Sconosciuto deve poter inserire la propria password	Soddisfatto
R-4-F-O	L'Utente Sconosciuto deve poter annullare l'accesso alla piattaforma	Soddisfatto
R-5-F-O	L'Utente Sconosciuto deve visualizzare un messaggio di errore nel caso di credenziali errate	Soddisfatto
R-16-F-O	L'Utente Registrato deve poter inserire un repository pubblico al sistema	Soddisfatto
R-17-F-D	L'Utente Registrato deve poter inserire un repository privato al sistema	Non soddisfatto
R-18-F-D	L'Utente Registrato deve poter inserire il Personal Access Token[G] collegato al proprio account	Non soddisfatto
R-19-F-D	L'Utente Registrato deve ricevere un messaggio di errore in caso di token inserito non valido	Non soddisfatto
R-20-F-O	L'Utente Registrato deve poter annullare l'inserimento di un repository	Soddisfatto
R-21-F-O	L'Utente Registrato deve poter inserire l'URL del repository	Soddisfatto
R-22-F-O	L'Utente Registrato deve ricevere un messaggio di errore nel caso di URL non valido	Soddisfatto
R-23-F-O	L'Utente Registrato deve poter visualizzare la lista dei propri repository personali	Soddisfatto
R-24-F-O	L'Utente Registrato deve poter selezionare una preferenza sulla tipologia di repository che desidera visualizzare	Soddisfatto
R-25-F-O	L'Utente Registrato deve poter visualizzare il nome di un repository	Soddisfatto
R-26-F-D	L'Utente Registrato deve poter visualizzare il nome del progetto associato al repository	Non soddisfatto
R-27-F-D	L'Utente Registrato deve poter visualizzare l'indicatore di visibilità di un repository	Non soddisfatto

Codice	Descrizione	Stato
R-28-F-D	L'Utente Registrato deve poter visualizzare la lista dei progetti ai quali contribuisce	Non soddisfatto
R-29-F-D	L'Utente Registrato deve poter visualizzare il nome di un progetto	Non soddisfatto
R-30-F-D	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi sulla documentazione di un progetto	Non soddisfatto
R-31-F-D	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi sui test di un progetto	Non soddisfatto
R-32-F-D	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi OWASP di un progetto	Non soddisfatto
R-33-F-O	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi sulla documentazione di un repository	Soddisfatto
R-34-F-O	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi sui test di un repository	Soddisfatto
R-35-F-O	L'Utente Registrato deve poter visualizzare le statistiche dell'analisi OWASP di un repository	Soddisfatto
R-36-F-O	L'Utente Registrato deve visualizzare la lista dei file sui quali è stata proposta remediation a seguito di un'analisi sui test per un repository	Soddisfatto
R-9-F-D	L'Utente Registrato deve visualizzare la propria area personale	Soddisfatto
R-10-F-O	L'Utente Registrato deve poter visualizzare il proprio nome Utente Registrato	Soddisfatto
R-11-F-O	L'Utente Registrato deve poter visualizzare la propria mail[G]	Soddisfatto
R-12-F-D	L'Utente Registrato deve poter visualizzare il proprio ruolo	Non soddisfatto
R-139-F-D	L'Utente Registrato deve poter visualizzare la propria immagine del profilo	Soddisfatto
R-140-F-D	L'Utente Registrato deve poter visualizzare l'opzione di reindirizzamento al proprio profilo Github	Soddisfatto
R-141-F-D	L'Utente Registrato deve poter visualizzare le proprie competenze registrate nella piattaforma	Non soddisfatto

Codice	Descrizione	Stato
R-142-F-D	L'Utente Registrato deve poter essere reindirizzato nel proprio profilo Github	Non soddisfatto
R-13-F-O	L'Utente Registrato deve poter effettuare il logout dalla piattaforma	Soddisfatto
R-14-F-O	L'Utente Registrato deve poter annullare la procedura di logout dalla piattaforma	Soddisfatto
R-15-F-O	L'Utente Registrato deve visualizzare un messaggio di errore nel caso di errore durante l'esecuzione di un operazione	Soddisfatto
R-92-F-P	L'Utente Registrato deve poter cercare un repository in una barra di ricerca	Soddisfatto
R-85-F-P	L'Utente Registrato deve poter visualizzare un repository sulla piattaforma esterna GitHub	Soddisfatto
R-78-Q-O	Il Developer[G] deve potersi autenticare e ricevere il ruolo di «Developer»	Soddisfatto
R-6-F-D	Il Developer può sincronizzare il proprio account GitHub	Non soddisfatto
R-7-F-D	Il Developer può annullare l'autenticazione con il provider esterno GitHub	Non soddisfatto
R-8-F-D	Il Developer deve poter annullare la sincronizzazione del proprio account GitHub	Non soddisfatto
R-37-F-O	Il Developer deve visualizzare la lista dei file sui quali è stata proposta remediation a seguito di un'analisi OWASP per un repository	Soddisfatto
R-38-F-O	Il Developer deve visualizzare la lista dei file sui quali è stata proposta remediation a seguito di un'analisi sulla documentazione per un repository	Soddisfatto
R-39-F-O	Il Developer deve essere in grado di visualizzare il dettaglio del file sul quale è stato proposto un cambiamento	Soddisfatto
R-40-F-D	Il Developer deve poter cambiare il path di destinazione del file sul quale è stata proposta una remediation	Non soddisfatto
R-41-F-D	Il Developer deve poter inserire il nuovo path all'interno del sistema	Non soddisfatto

Codice	Descrizione	Stato
R-42-F-D	Il Developer deve poter annullare il cambio di path di un file nel contesto di una proposta di remediation	Non soddisfatto
R-43-F-D	Il Developer deve visualizzare un messaggio di errore nel caso abbia inserito un path non valido per un file, nel contesto di una proposta di remediation	Non soddisfatto
R-44-F-D	Il Developer deve poter accettare una proposta di remediation	Non soddisfatto
R-45-F-D	Il Developer deve ricevere una notifica una volta che è stata inviata con successo una pull request sul repository GitHub	Non soddisfatto
R-46-F-D	Il Developer deve poter annullare l'accettazione di una proposta di remediation	Non soddisfatto
R-47-F-D	Il Developer deve ricevere un messaggio di errore nel caso di fallimento dell'accettazione della remediation	Non soddisfatto
R-48-F-D	Il Developer deve essere informato se la remediation proposta non è più coerente con lo stato del sistema, e quindi non può essere effettuata	Non soddisfatto
R-49-F-O	Il Developer deve poter eliminare un repository dal sistema	Soddisfatto
R-50-F-O	Il Developer deve poter annullare l'operazione di eliminazione di un repository dal sistema	Soddisfatto
R-51-F-D	Il Developer deve poter visualizzare la lista dei procedimenti in corso all'interno del sistema	Non soddisfatto
R-52-F-D	Il Developer deve visualizzare il nome del repository sul quale sta avvenendo il procedimento	Non soddisfatto
R-53-F-D	Il Developer deve visualizzare la data di avvio di ogni elemento dalla lista dei procedimenti in corso in corso.	Non soddisfatto
R-54-F-D	Il Developer deve visualizzare il nome branch aperto di ogni elemento della lista delle remediation in corso.	Non soddisfatto
R-55-F-D	Il Developer deve poter interrompere una remediation avviata	Non soddisfatto
R-56-F-D	Il Developer deve visualizzare un messaggio di avvenuta interruzione di una remediation in corso, in caso ne abbia richiesto la stessa	Non soddisfatto

Codice	Descrizione	Stato
R-57-F-D	Il Developer deve poter annullare un'analisi in corso all'interno del sistema	Non soddisfatto
R-58-F-D	Il Developer deve visualizzare un messaggio di avvenuto annullamento di un'analisi in corso	Non soddisfatto
R-59-F-D	Il Developer deve visualizzare una lista con le ultime analisi portate a termine	Non soddisfatto
R-60-F-D	Il Developer deve visualizzare il nome del repository sul quale un'analisi è stata terminata	Non soddisfatto
R-61-F-D	Il Developer deve visualizzare la data di completamento di un'analisi terminata	Non soddisfatto
R-62-F-D	Il Developer deve poter visualizzare la specifica di un'analisi portata a termine all'interno di un repository, che non sia l'ultima	Non soddisfatto
R-63-F-D	Il Developer, nel contesto della visualizzazione di un'analisi passata, deve poter visualizzare il grafico dell'analisi sulla documentazione	Non soddisfatto
R-64-F-D	Il Developer, nel contesto della visualizzazione di un'analisi passata, deve poter visualizzare il grafico dell'analisi sui test	Non soddisfatto
R-65-F-D	Il Developer, nel contesto della visualizzazione di un'analisi passata, deve poter visualizzare il grafico dell'analisi OWASP	Non soddisfatto
R-66-F-D	Il Developer, nel contesto della visualizzazione di un'analisi passata, deve poter visualizzare le proposte di remediation elaborate in seguito all'analisi	Non soddisfatto
R-67-F-D	Il Developer, nel contesto della visualizzazione di un'analisi passata, deve poter visualizzare la proposta di remediation specifica inerente ad un singolo file	Non soddisfatto
R-69-F-D	Il Developer deve poter consultare l'ultima analisi effettuata all'interno di un repository	Soddisfatto
R-70-F-D	Il Developer deve poter visualizzare la data di fine di un'analisi nella lista delle analisi passate relative ad un repository	Non soddisfatto
R-71-F-D	Il Developer deve poter visualizzare la lo stato di un'analisi nella lista delle analisi passate relative ad un repository	Non soddisfatto

Codice	Descrizione	Stato
R-72-F-O	Il Developer deve poter avviare un'analisi generale all'interno del repository	Soddisfatto
R-73-F-D	Il Developer deve poter avviare un'analisi sui test all'interno del repository	Non soddisfatto
R-74-F-D	Il Developer deve poter avviare un'analisi sulla documentazione all'interno del repository	Non soddisfatto
R-75-F-D	Il Developer deve poter avviare un'analisi OWASP all'interno del repository	Non soddisfatto
R-76-F-D	Il Developer deve ricevere un messaggio di errore nel caso si verificasse un conflitto con un'altra analisi all'avvio di un'analisi	Non soddisfatto
R-77-F-O	Il Developer deve poter annullare l'avvio di un'analisi	Soddisfatto
R-79-Q-D	Il Project Manager[G] deve potersi autenticare e ricevere il ruolo di «Project Manager»	Non soddisfatto
R-80-F-D	Il Project Manager deve poter visualizzare la lista dei propri progetti	Non soddisfatto
R-81-F-D	Nella lista dei progetti, il Project Manager deve poter visualizzare il grafico percentuale sull'analisi della documentazione	Non soddisfatto
R-82-F-D	Nella lista dei progetti, il Project Manager deve poter visualizzare il grafico percentuale sull'analisi dei test	Non soddisfatto
R-83-F-D	Nella lista dei progetti, il Project Manager deve poter visualizzare il grafico percentuale sull'analisi della correttezza OWASP	Non soddisfatto
R-84-F-D	Il Project Manager deve poter visualizzare i dettagli di un progetto	Non soddisfatto
R-87-F-D	Il Project Manager deve poter creare un nuovo progetto	Non soddisfatto
R-88-F-D	Il Project Manager deve poter inserire il nome del progetto	Non soddisfatto
R-89-F-D	Il Project Manager deve poter modificare il nome di un progetto	Non soddisfatto

Codice	Descrizione	Stato
R-90-F-D	Il Project Manager deve ricevere un errore se il nuovo nome del progetto che si sta inserendo è già presente per quell'utente	Non soddisfatto
R-91-F-D	Il Project Manager deve poter aggiungere un repository al progetto	Non soddisfatto
R-93-F-D	Il Project Manager deve poter visualizzare una lista di repository da poter aggiungere al progetto	Non soddisfatto
R-94-F-D	Il Project Manager deve poter annullare l'operazione di aggiunta repository al progetto	Non soddisfatto
R-95-F-D	Il Project Manager deve poter aggiungere dei developer al progetto	Non soddisfatto
R-96-F-P	Il Project Manager deve poter cercare il nome di un developer su una barra di ricerca	Non soddisfatto
R-97-F-D	Il Project Manager deve poter visualizzare una lista di developer da poter aggiungere al progetto	Non soddisfatto
R-98-F-D	Il Project Manager deve poter annullare l'aggiunta dei developer al progetto	Non soddisfatto
R-99-F-D	Il Project Manager deve poter visualizzare il nome e la foto profilo di un developer	Non soddisfatto
R-100-F-D	Il Project Manager deve poter visualizzare e gestire gli attuali membri del team	Non soddisfatto
R-101-F-D	Il Project Manager deve poter rimuovere un developer dal progetto	Non soddisfatto
R-102-F-D	Il Project Manager deve poter annullare la rimozione di un developer dal progetto	Non soddisfatto
R-103-F-D	Il Project Manager deve poter visualizzare le competenze del team su un progetto	Non soddisfatto
R-104-F-D	Il Project Manager deve poter visualizzare il profilo di un membro del team (developer)	Non soddisfatto
R-105-F-D	Il Project Manager deve poter visualizzare il profilo GitHub di un membro del team (developer)	Non soddisfatto

Codice	Descrizione	Stato
R-106-F-D	Il Project Manager deve poter visualizzare le icone delle tecnologie di competenza, accanto al nome dei developer	Non soddisfatto
R-107-F-D	Il Project Manager deve poter visualizzare le statistiche di contribuzione di un membro del team	Non soddisfatto
R-108-F-D	Il Project Manager deve poter visualizzare i linguaggi di programmazione maggiormente utilizzati da un membro del team	Non soddisfatto
R-110-F-D	Il Project Manager deve poter visualizzare il numero di commit effettuati da un membro del team	Non soddisfatto
R-111-F-D	Il Project Manager deve poter visualizzare il numero di pull request gestite da un membro del team	Non soddisfatto
R-112-F-D	Il Project Manager deve poter visualizzare lo stack tecnologico utilizzato in un progetto	Non soddisfatto
R-113-F-D	Il Project Manager deve poter visualizzare l'elenco di framework e librerie, utilizzare nello stack tecnologico di un progetto	Non soddisfatto
R-114-F-D	Il Project Manager deve poter visualizzare l'elenco di segnalazioni a: framework e librerie con rischi di sicurezza o deprecate, utilizzare nello stack tecnologico di un progetto	Non soddisfatto
R-115-F-D	Il Project Manager deve poter visualizzare suggerimenti generati dall'IA per la sostituzione di framework e librerie, utilizzate nello stack tecnologico di un progetto	Non soddisfatto
R-139-F-D	Il Project Manager deve poter eliminare un progetto di sua proprietà	Non soddisfatto
R-140-F-D	Il Project Manager deve poter annullare l'eliminazione di un progetto di sua proprietà	Non soddisfatto
R-116-Q-P	Il Business Owner[G] deve poter visualizzare la lista dei propri progetti	Non soddisfatto
R-117-F-P	Il Business Owner deve poter visualizzare il budget complessivo per il proprio progetto	Non soddisfatto
R-118-F-P	Il Business Owner deve visualizzare le spese affrontate per un progetto	Non soddisfatto

Codice	Descrizione	Stato
R-119-F-P	Il Business Owner deve poter comparare il budget a disposizione con le spese affrontate	Non soddisfatto
R-120-F-P	Il Business Owner deve poter visualizzare il dettaglio di un progetto	Non soddisfatto
R-121-F-P	Il Business Owner deve poter visualizzare il nome di un progetto	Non soddisfatto
R-122-F-P	Il Business Owner deve poter cambiare il budget complessivo per un progetto	Non soddisfatto
R-123-F-P	Il Business Owner deve poter annullare il cambiamento del budget complessivo	Non soddisfatto
R-124-F-P	Il Business Owner deve poter cambiare la spesa sostenuta per un progetto	Non soddisfatto
R-125-F-P	Il Business Owner deve poter annullare il cambiamento della spesa sostenuta per un progetto	Non soddisfatto
R-126-F-P	Il Business Owner deve poter visualizzare il nome del Project Manager di un progetto	Non soddisfatto
R-127-F-P	Il Business Owner deve poter visualizzare la mail di contatto del Project Manager di un progetto	Non soddisfatto
R-128-F-P	Il Business Owner deve poter visualizzare la data di creazione di un progetto	Non soddisfatto
R-129-F-P	Il Business Owner deve poter visualizzare la lista delle tecnologie utilizzate in un progetto	Non soddisfatto
R-130-F-P	Il Business Owner deve visualizzare, per ogni elemento di una lista di stack tecnologico, il logo della tecnologia	Non soddisfatto
R-131-F-P	Il Business Owner deve visualizzare, per ogni elemento di una lista di stack tecnologico, il nome della tecnologia	Non soddisfatto
R-109-F-P	Il Business Owner deve visualizzare, per ogni elemento di una lista di stack tecnologico, l'utilizzo che si fa della tecnologia	Non soddisfatto
R-132-F-P	Il Business Owner deve poter visualizzare la lista del team di sviluppo che fa parte di un progetto	Non soddisfatto
R-133-F-P	Il Business Owner, per ogni elemento della lista del team di sviluppo, deve visualizzare il nome dello sviluppatore[G]	Non soddisfatto

Codice	Descrizione	Stato
R-134-F-P	Il Business Owner, per ogni elemento della lista del team di sviluppo, deve visualizzare il ruolo dello sviluppatore all'interno di un progetto	Non soddisfatto
R-135-F-P	Il Business Owner deve poter visualizzare i dettagli di un developer che lavora ad un suo progetto	Non soddisfatto
R-136-F-P	Il Business Owner deve poter visualizzare il numero di progetti ai quali lo sviluppatore ha preso parte	Non soddisfatto
R-137-F-P	Il Business Owner deve poter visualizzare le tecnologie che uno sviluppatore può utilizzare	Non soddisfatto
R-138-F-P	Il Business Owner deve poter visualizzare i ruoli che uno sviluppatore può assumere	Non soddisfatto

5.2. Grafici di stato dei requisiti

5.2.1. Requisiti funzionali

Sono stati individuati 142 requisiti funzionali. Di questi sono stati soddisfatti 35.

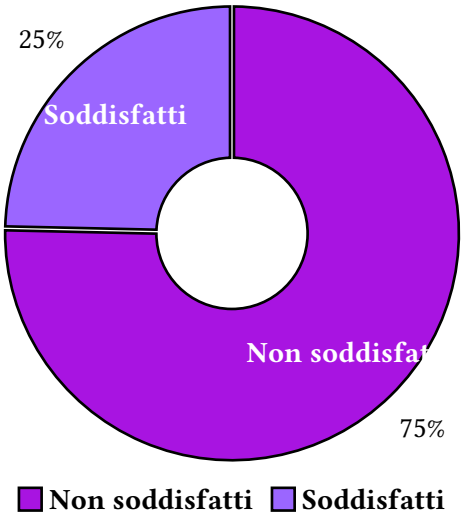

Figura 17: Requisiti funzionali soddisfatti

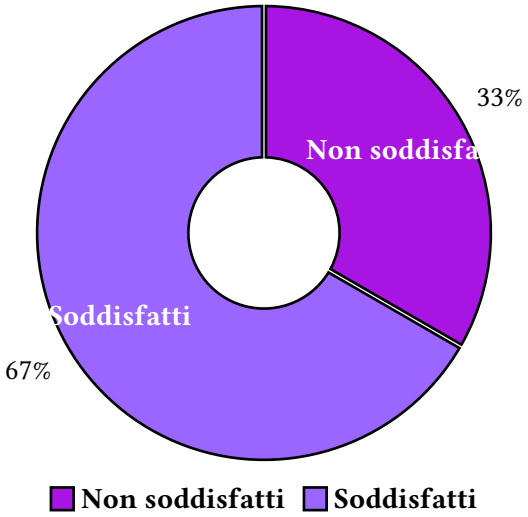

Figura 18: Requisiti funzionali soddisfatti
Utente generico

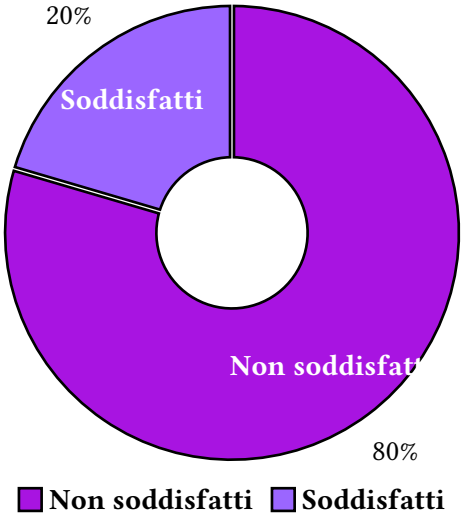

Figura 19: Requisiti funzionali soddisfatti Developer

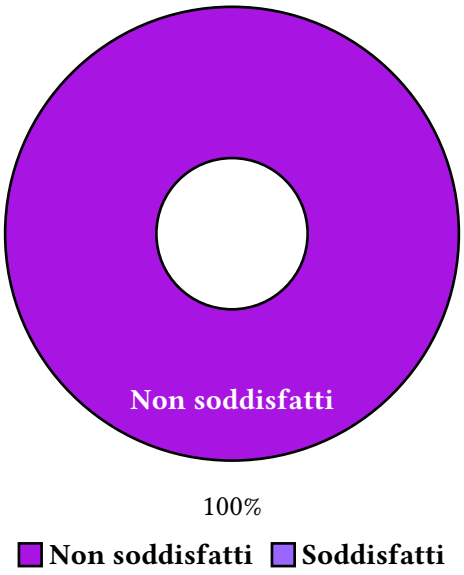

Figura 20: Requisiti funzionali soddisfatti Project Manager

100%

Non soddisfatti **Soddisfatti**

Figura 21: Requisiti funzionali soddisfatti Business Owner

5.2.2. Requisiti funzionali obbligatori

Tutti i requisiti obbligatori sono stati soddisfatti.

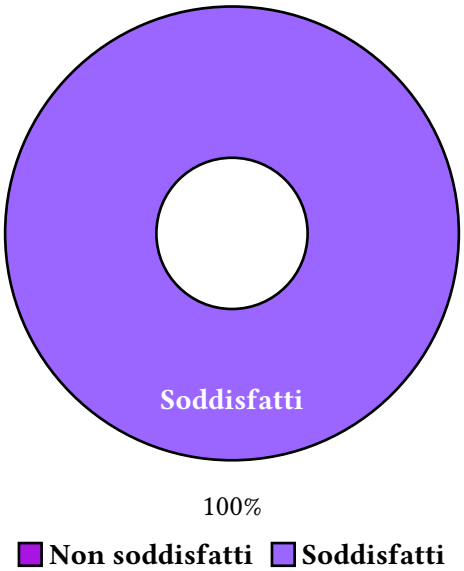

Figura 22: Requisiti funzionali obbligatori soddisfatti

5.2.3. Requisiti funzionali desiderabili

Sono stati soddisfatti quattro requisiti desiderabili su novanta.

Figura 23: Requisiti funzionali desiderabili soddisfatti

5.2.4. Requisiti funzionali opzionali

Sono stati soddifatti due requisiti opzionali su ventisette.

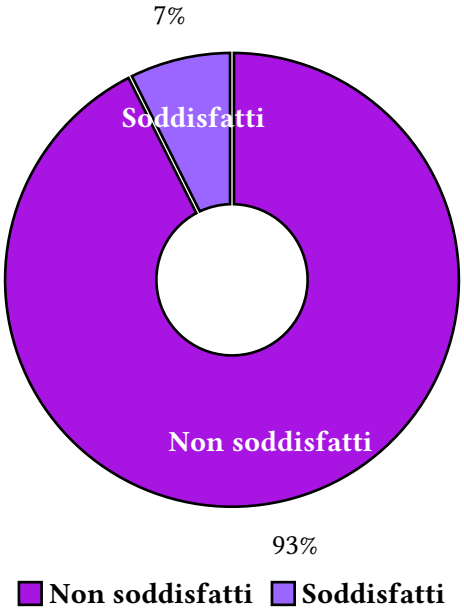

Figura 24: Requisiti funzionali opzionali soddisfatti